

HISTORY OF SQL

1] SQL is also known as SEQUEL

2] The SQL or SEQUEL was invented in year 1979 by IBM company

3] In the year 1981, the oracle company has renamed SEQUEL as SQL

4] The person called as E.F.CODD has renamed as SEQUEL as SQL.

5] The father of SQL :-

1] F. Boyce

2] D. Chamberlin.

6] SQL is non case sensitive lang.

Types of Application

Defination : The set of instruction which are used to perform an any specific task is known as applications.

Types :
 1] Stand Alone Application
 2] Web Application
 3] Client/Server Application

1] Stand Alone Application :

The Application which does not required on any internet connectively to access it are known as stand alone application

- 1] The internet connection is not required
- 2] Database is optional part
- 3] Multiple user can't able to access at a same time.
- 4] Installation is Mandatory
- 5] eg :- camera, calendar, calculator, paint, Notepad etc.

2] Web Application :

The application which we can access through on any search engine are known as web applications.

- 1] Internet connection is required.
- 2] Database is Mandatory part
- 3] The multiple user can able to access the same feature at a time.

4] Installation is not mandatory part

5] Eg: WWW.insta.com

WWW.facebook.com

3] client-server-application:

The application which we can able to access through the client-server these type of application is known as client-server application

1] Internet connection is required

2] Database is mandatory part

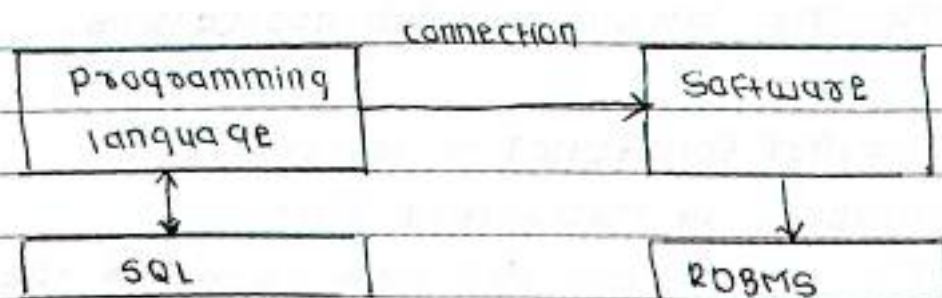
3] Multiple user can't able to access the same time feature at a time

4] Installation is mandatory

5] Eg: Banking application, gaming application

SQL

SQL is the backend technology which we can consider as one of the programming language which is used to interact with the software. called as Relational Database Management System (RDBMS).



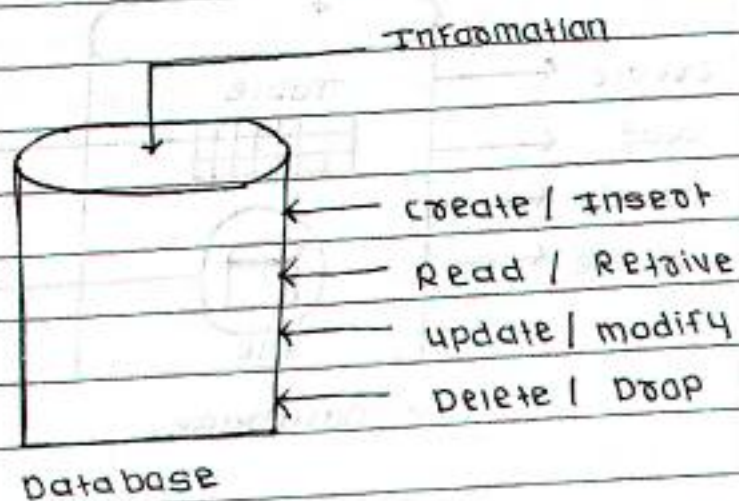
Relational Database management system :-

Database (DB) :-

Database is a place in which we can store our information in systematic and organized manner is known as Database (DB).

1] on DB the user can able to perform the four basic operations.

- 1] create or insert
- 2] Read or Retrive
- 3] update or Drop Modify
- 4] Read or Delete or Drop



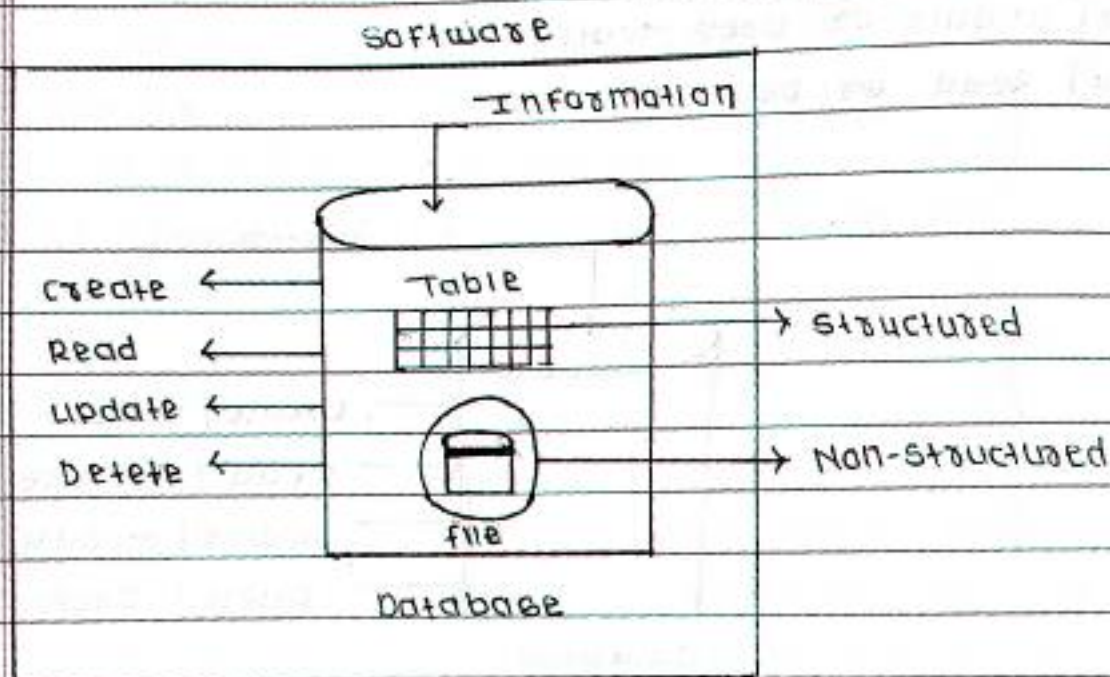
2] These are the 4 basic operation are also called as 'CRUD' operations

□ DBMS :-

1] DBMS is a SW which is used to maintain and manage the database by providing the multiple features.

2] The DBMS software will store the data in the form of either table format or file format.

3] The table format is also known as structured format and file format also known as non-structured format (unorganized manner)

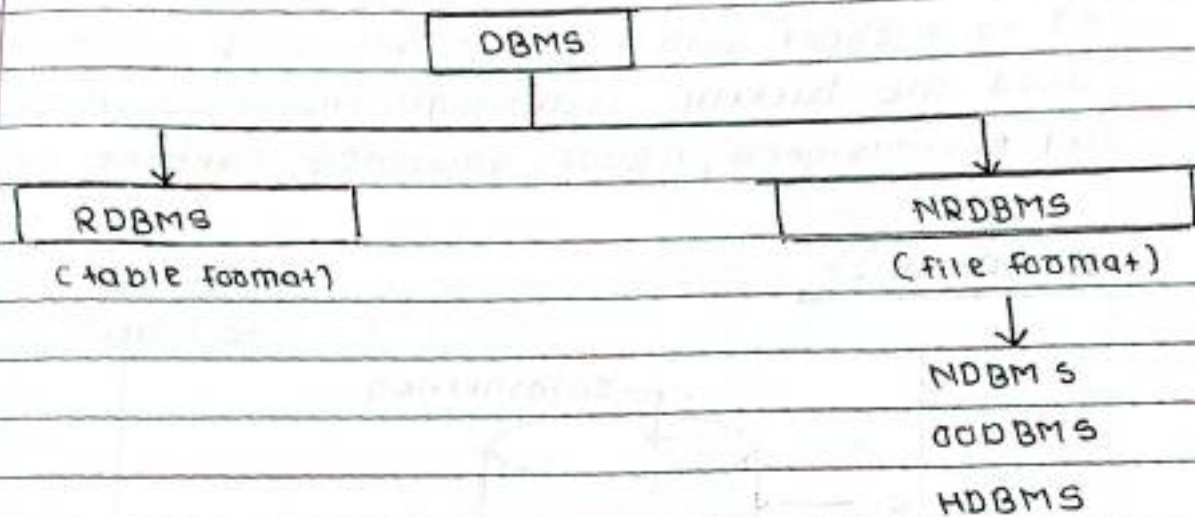


DBMS.

4] To interact with DBMS software the user can able to use either SQL language or NoSQL language.

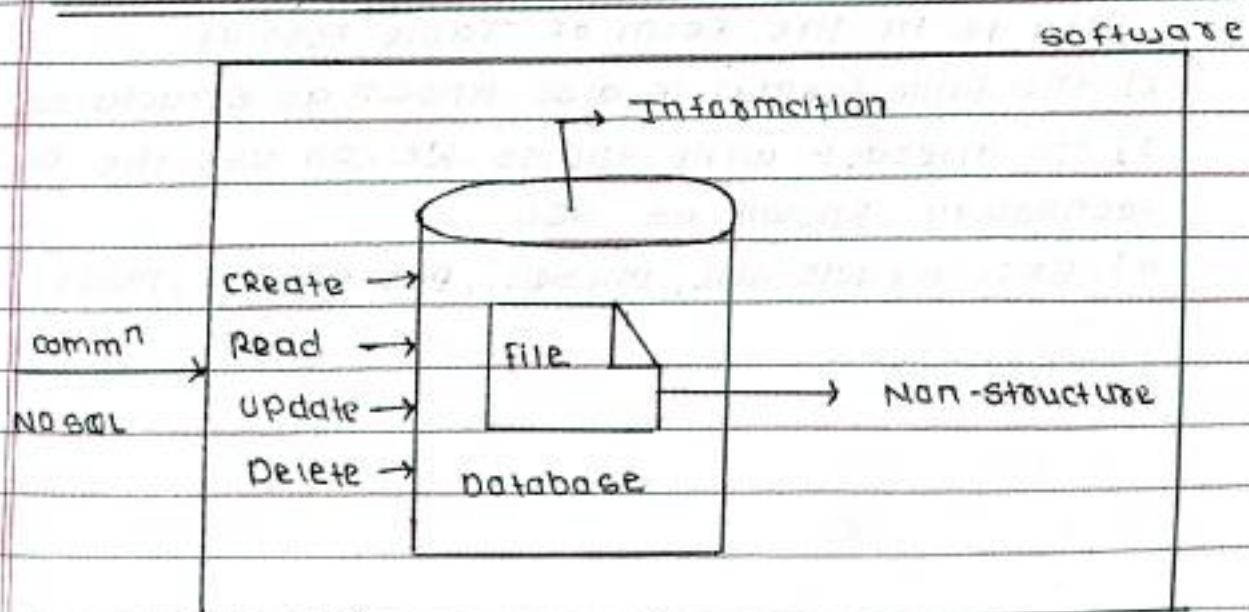
5] Based on the format in which DBMS SW will store info. according to format we need to select any one backend technology i.e SQL or NoSQL.

- 6] The DBMS Software is classified into two types :-
- 1] Relational Database management system (table)
 - 2] Non-Relational Database management system (file format)



- 7] NRDBMS further divide into :-
- Network db management
 - object oriented database management system
 - Hierarchy based database management system.

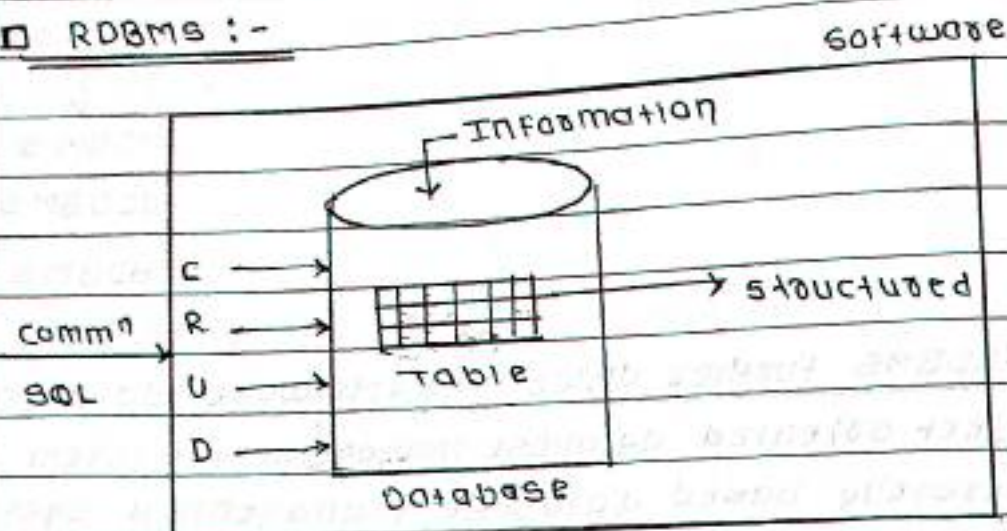
□ NDBMS / Non Relational DB management system.



NRDBMS

- 1] It is a type of DBMS SW which is used to maintain and manage the database by storing the complete data in the form of file format.
- 2] The file format is also known as non-structured.
- 3] To interact with the non relational DBMS we can use the backend technology known as No-SQL.
- 4] Ex:- MongoDB, HBase, AmazonDB, Firebase etc.

RDBMS :-

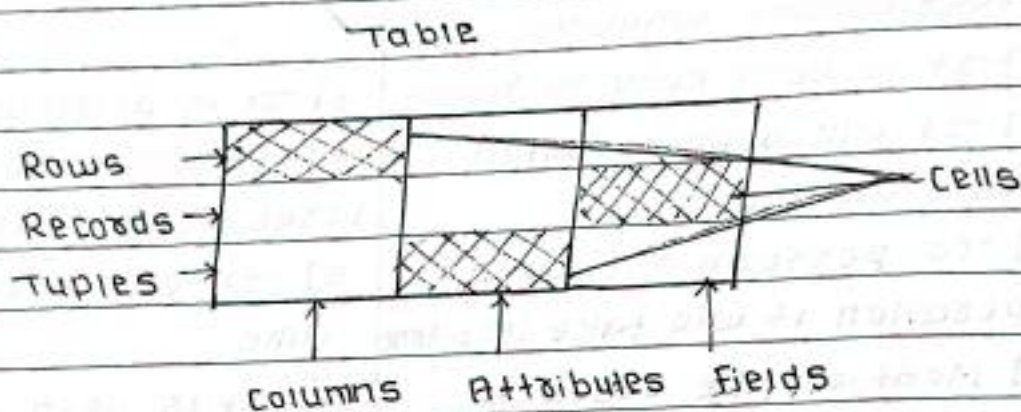


- 1] It is the type of DBMS software which is used to maintain and manage the DB by storing the complete data in the form of Table format.
- 2] The table format is also known as structured.
- 3] To interact with RDBMS we can use the backend technology known as SQL.
- 4] Ex:- Oracle SQL, MySQL, PostgreSQL, MSOL, TSOL etc.

RDBMS	NDBMS
1] RDBMS Will support table format	1] It will support file format
2] To interact with RDBMS we have to use SQL as a programming language	2] No SQL
3] It is very easy to handle	3] It is difficult to handle
4] It will support small scale dataset.	4] It will support for large scale dataset.
5] To perform the CRUD operation it will take less time	5] It will take more time.
6] Most of the cases to handle text related information we need to use RDBMS	6] It is used to handle the multimedia type of information
7] Schema validation is mandatory in RDBMS	7] Schema validation is optional in NDBMS
8] The RDBMS will support ACID property (Atomic, Consistency, Isolation, Durability)	8] It will support both ACID and CAP theorem ^{tolerance} (Consistency, Availability, Partition)
9] The RDBMS will support only vertical scaling	9] It will support both vertical and Horizontal scaling
10] The RDBMS consist of relation, it will support for complex query known as join and subqueries	10] It will not support for Join and sub-queries.
11] In RDBMS, the complete schema is rigid one.	11] The schema will not be having any structure.
12] Example : SQL, MySQL, My SQL . etc	12] Example : MongoDB, firebase, Hbase, Amazon DB. etc.

• Description of table and rules of table / SQL / EF, CDD.

Table : The logical arrangement of rows and columns are known as tables



Column : The logical arrangement of all the vertical lines on the table are known as columns.

The alternative names of columns are :

1] Attributes

2] Fields

Rows : The logical arrangement of all the horizontal lines in the tables are known as rows.

The alternative names of Rows are :

1] Records

2] Tuples.

Cells :- The small unit of a table in which the data will get store that unit is called as cell.

$$\text{No of cells} = \text{No of rows} \times \text{No of columns}$$

Either in SQL or Excel whatever the CRUD operation the user is going to perform directly the operation will affect on the respective cell

RULES OF EF CODD :-

- 1] The data entered into cell must be single valued data (Atomic Data)

SID	SNAME	MNO		SID	SNAME	MNO	A/MNO
01	SHAM	108		01	SHAM	108	NULL
02	RAM	107,106		02	RAM	107	106

(Invalid table)

(valid table)

- 2] In SQL or In RDBMS we are dealing with the table, the table is also known as schema. We must have to validate the each and every column of the table by using data type and constraints.

- 3] In RDBMS it is difficult to handle the multimedia type of information (It is not suitable for multimedia) We can easily handle upto 4GB multimedia type of data in each cell.

If we store any multimedia type of data in table the SQL compiler will detect the extra information, the of a multimedia. that extra information known as metadata.

The metadata the SQL compiler will manage in the form of table format.

STD

SID	SNAME	IMAGE
01	RAM	IMG
02	SHAM	IMG

Name : R-123
 Size : 56KB
 Res : 360x480
 format : JPG → Metadata.

name	Size	Resolution	Format	
R-123	56	360x480	JPG	} Metatable
S-123	100	360x480	PNG	

The details of another data is known as metadata. The metadata will get create by compiler itself. The metadata will get store in the form of table format that table is known as metatable.

4] In SQL we can distribute the same unit data into multiple tables and if it is required we can easily make the connection betⁿ all the multiple tables with the help of key attributes (foreign key).

(key attribute) KA

EMPLOYEE

EMP	ENAME	SAL	DNO	CID
E1	RAM	10000	10	C1
E2	SHAM	20000	10	C1
E3	BARBARA	50000	20	C2
E4	RAJU	150	Null	Null
E5	SHET	300	20	C2

DEPT

DNO	DNAME	LOC
10	HR	Pune
20	Sales	Hyd

PARENT TABLE (T2)

CHILD

CID	CNAME	AGE
C1	KARAN	201
C2	ARJUN	203

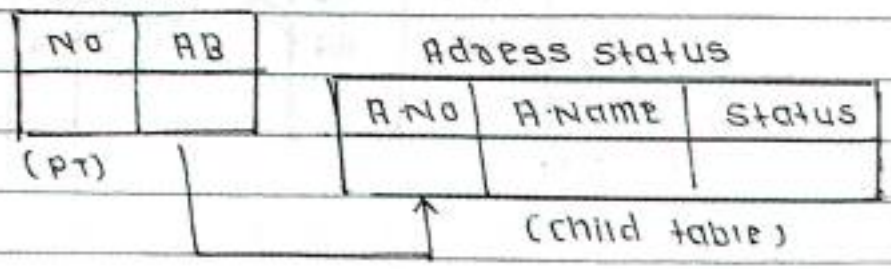
(T1) child table

Cardinality : is the no. of unique val in column or the no. of relationship betⁿ table. PARENT TABLE (T3)

In SQL there are three types of relationships (cardinality)

1) one to one :- The one record from the parent table if it is referring to only one record in the child table this type of relationship known as one to one relationship.

Ex:- Account



2] one to many / many to one

The one record from the parent table if it is referring the multiple records to the child table. These type of relationship known as one to many.

Ex: Department table $\rightarrow$ employee table.

3] many to many

The one record from the first table if it is referring to multiple record in the second table, and the one record from the second table if it is also referring to multiple record in the first table. This type of relationship in SQL is known as many to many.

Ex: Actor table Movie table

A1	A1	M1	Karan and Arjun
BK	A2	M2	Phir huxa pheri
SRK	A3	M3	Solay
HRX	A4	M4	DDLJ

Schema Validation process

- It is the process of verifying / validating ^{whether} the proper data is entering in the column or not.
- The schema validation process we can do by using data types or by using constraints (Rules).

Datatypes :-

Def :- Datatypes are used to determine which type of data should be stored into the particular memory location.

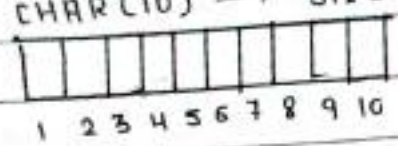
- Each column from the table we need to consider as a variable.
- In SQL there are 5 types of data types :-
 - CHAR() → character
 - VARCHAR() → Variable character or VARCHAR2()
 - DATE
 - Large object
 - character large object (CLOB)
 - Binary large object (BLOB)
 - NUMBER()

1) CHAR() :-

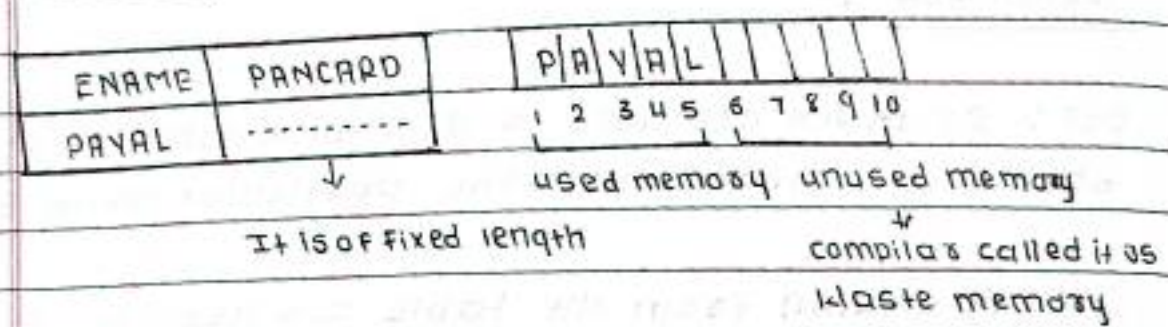
- The CHAR() datatype will allow uppercase 'A' to 'Z' any character and lowercase 'a' to 'z' any character, Alphanumeric and any type of special characters and it will also allow 0 to 9 any number.

Syntax :- CHAR (size)

Ex:- CHAR(10) → size is 10 then it allocate 10 memory block
max capacity of length is 2000



- The char data type will follow the fixed length of memory allocation.

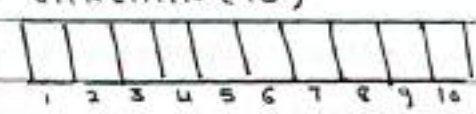


VARCHAR () Datatype :-

- It is very similar to char datatype.
- The VARCHAR Datatype will also allow upper case A to Z any characters, lowercase a to z any character. Alphanumerics and any type of special character and also allow 0 to 9 any number.

Syntax :- VARCHAR (size) | max capacity of VARCHAR is 2000

Ex:- VARCHAR (10)



- The VARCHAR Datatype will follow variable length of memory allocation.
- If we not use VARCHAR in variable length of data then performance, efficiency issue happen.

ENAME	PAN										
RATU		R	A	T	U						
		1	2	3	4	5	6	7	8	9	10

used memory free memory → the empty memory given back to actual memory

- It is also used to save the memory.
- The updated version of VARCHAR datatype is VARCHAR2()
- It is similar to VARCHAR datatype

• Syntax :- VARCHAR2(size) → max length is upto 4000 (no. of blocks)

Ex: VARCHAR2(10)

Large object Datatype :-

- It is used to store the huge amount of data
 - 1] The max capacity of large object it is upto 4GB
 - 2] The large object has been classified into: character large object and second is binary large object.
 - 1] character large object
 - 2] binary large object

1] character large object :-

The character large object is used to store the huge amount of character type of information.

Syntax: CLOB

Imp ⇒ { The Note :- 1GB = 1.074 Million character
4GB = 5.37 Millions character }

2) Binary large object :-

It is used to store multimedia type of information

Syntax :- BLOB

DATE Datatype :-

- These DATATYPE is used to stored any DATE related information in any specific format

Syntax :- DATE

- In SQL to stored the DATE related information the ORACLE has provided the two diff. format :-

1) DD-MON-YY

[Mon means first 3 char of month]

2) DD-MON-YYYY

EX:- 1) → 12-FEB-26 2) → 12-FEB-2026

NUMBER Datatype :-

It is used to store an any number related information.

Syntax :- Number (precision, [scale])

↓ ↓
 used to handle optional part
 integer part (handle decimal part)

- precision :-

It is used to determine the no-of digits which are required to store an any integer value.

The range of precision is 1 to 38.

The default value is 38.

• Scale :

- It is used to determine the no. of digit which are required to store an any decimal value within the precision.
- The range of scale is -84 to 127
- The default value of scale is 0

Examples :- 1] Number (4) :

9	9	9	9
---	---	---	---

 -9999 to +9999

2] Number (4,1) :

9	9	9	.	9
---	---	---	---	---

 -999.9 to +999.9
└─ point is here bcz scale is 1.

3] Number (3,5) :

0	0	9	9	9
---	---	---	---	---

 { scale - precision }
5 4 3 2 1 = 5 - 3 = 2

4] Number (2,5) :

0	0	0	9	9
---	---	---	---	---

 = there are chance of 2 zeros.

Examples :-

1] Number (4) → S=0 :

9	9	9	9
---	---	---	---

 -9999 to +9999

2] Number (5,3) → P>S :

9	9	.	9	9
---	---	---	---	---

 -999.99 to +99.999
5 4 3 2 1

3] Number (3,3) → P=S :

9	9	9
---	---	---

 -0.999 to +0.999

4] Number (3,6) → P<S / S>P :

0	0	0	.	9	9	9
---	---	---	---	---	---	---

 -0.000999 to +0.000999
↓
more chances of 0's

5] Number (2,7) → P<S :

0	0	0	0	0	9	9
---	---	---	---	---	---	---

6] Number (4,-2) → P>S :

9	9	9	9	.	0	0
---	---	---	---	---	---	---

↓
add zero after result
if (-) comes

7] Number (6,-3) :

9	9	9	9	9	9	.	0	0	0
---	---	---	---	---	---	---	---	---	---

constraints

- constraints are the rules which are used to validate the schema in the database.
- In SQL there are 7 types of constraints :-
 - 1] unique
 - 2] null vs not null
 - 3] check constraints
 - 4] primary key
 - 5] Foreign key
 - 6] Default constraints
 - 7] Auto increment

1] unique constraint

This constraint is used to avoid the repeated values or the duplicate values from the respective column.

Syntax :- unique

2] null vs not null

It is used to consider the respective column as optional

Syntax :- Null

Not null constraint is used to consider the respective column as mandatory column

Syntax :- Not Null

3] check constraints :-

This constraint is used to validate the column of data by providing the extra conditions.

Syntax :- CHECK (condition)



4] primary key :-

This constraint is used to identify the unique records from the table.

Syntax : Primary key

The primary key is combination of unique and not null cons. The primary key will not allow any repeated records and any empty cells.

In a same table we need to have only one primary key

5] Foreign key :-

The constraint which is used to establish the connection betⁿ all the multiple table is known as foreign key

Syntax : Foreign key

It is not the combination of unique and not null

The foreign key will allow repeated values and even empty cells

In a same table we can able to have upto 255 foreign key

The foreign key also known as referential Integrity constraint

6] Default constraint :-

This constraint is used to assign the default value to the respective column.

Syntax :- Default Value

7] Auto increment :-

This constraint is used to assign the auto increment feature to the respective column.

Syntax :- Auto increment (start value, skip v, end v)

Example :-

PK				CI			CI/ID			Client		
Not Null	Not Null	Null	Not Null				CID PK	CNAME	ADDRESS			
unique	(-)	(-)	unique				C1	KARAN	PART 1			
Number (3)	VARCHAR (50)	Number (5,1)	Number (10)	FK	FK		C2	ARJUN	DUBAI			
EMP	ENO	ENAME	SAL	MNO	DNO	CID	C3	SANTU	US			
101	SHAM	1000	123...980	D1	C1							
102	ADITYA	5000	123...981	D1	C1							
103	SIDDHI	1000	123...982	D2	C2							
104	SRK	150	123...983	D2	C2							
105	TEJAS	150	123...984	D1	C1							
106	SHIVAM	Null	123...988	Null	Null							

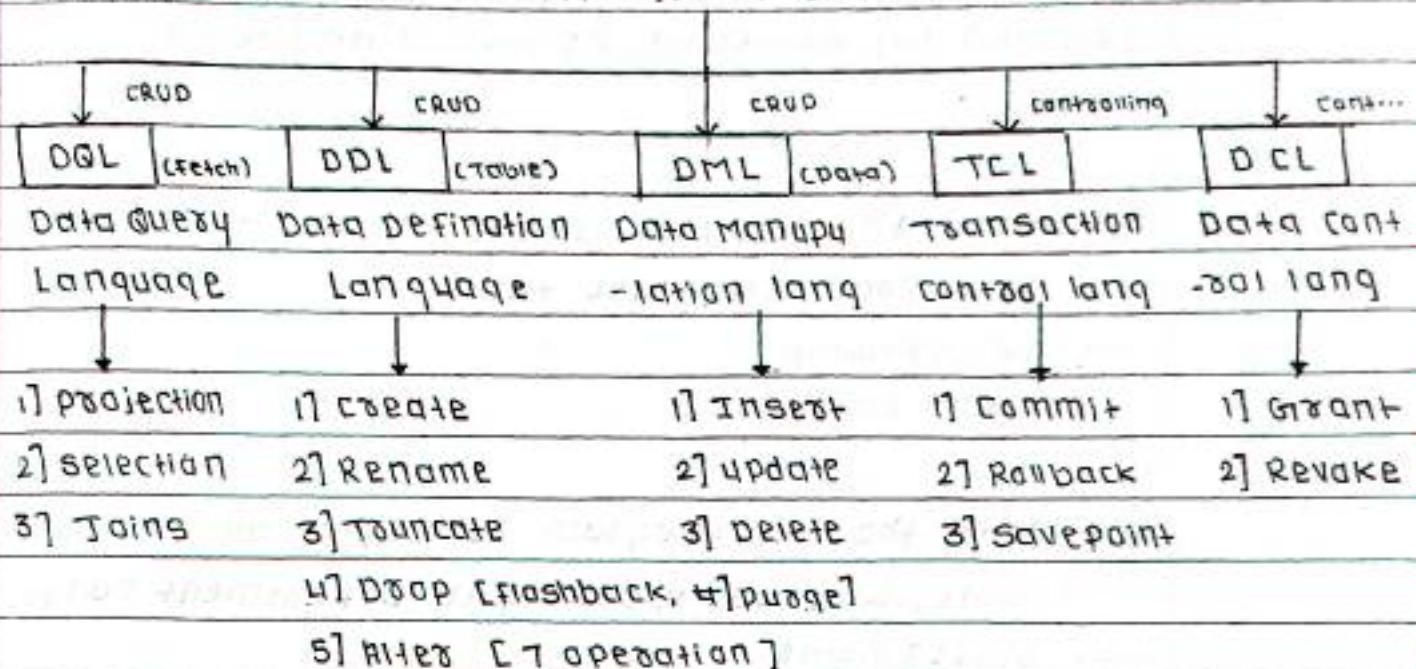
DNO PK	DNAME	Loc
D1	HR	pune
D2	SALES	Delhi
D3	Prod	KAR

CID PK	CNAME	ADDRESS
C1	KARAN	PART 1
C2	ARJUN	DUBAI
C3	SANTU	US

Statements in a SQL

statements are the collection of n of multiple keywords which are used to perform the CRUD operation in database

In SQL there are 5 diff. type of statements :-



DQL (Data Query Language)

Def:- This statement is used to fetch the data from the table or from the Database

In DQL there are three methods to Fetch the Data from the table :-

1] Projection

It is the process of fetching the data from table by selecting only the columns.

2] Selection

It is the process of fetching the data from table by selecting both rows and columns.

3] JOINS

It is the process of fetching the data from the multiple tables simultaneously is known as joins.

In the DDL there is a one keyword known as SELECT which is used to display data which has been selected by selection, projection and joins.

Q - Write the SQL program to display all the employee name from employee table.

→ `SELECT name
FROM employee;`

Q - Write the SQL program to display all the employee name, who are working in department no. 10.

→ `SELECT name
FROM employee
WHERE dept_no = 10;`

Q - Write SQL prog to display employee name, employee department name and client name for all the employees.

→ `SELECT e.name, d.dept_name, c.client_name
FROM employee;`

• Projection :- It is the process of fetching the data from table by or the database by selecting only the column.

Syntax :- * / [Distinct] col-name / Expression [alias]

1

2

3

from Table-name;

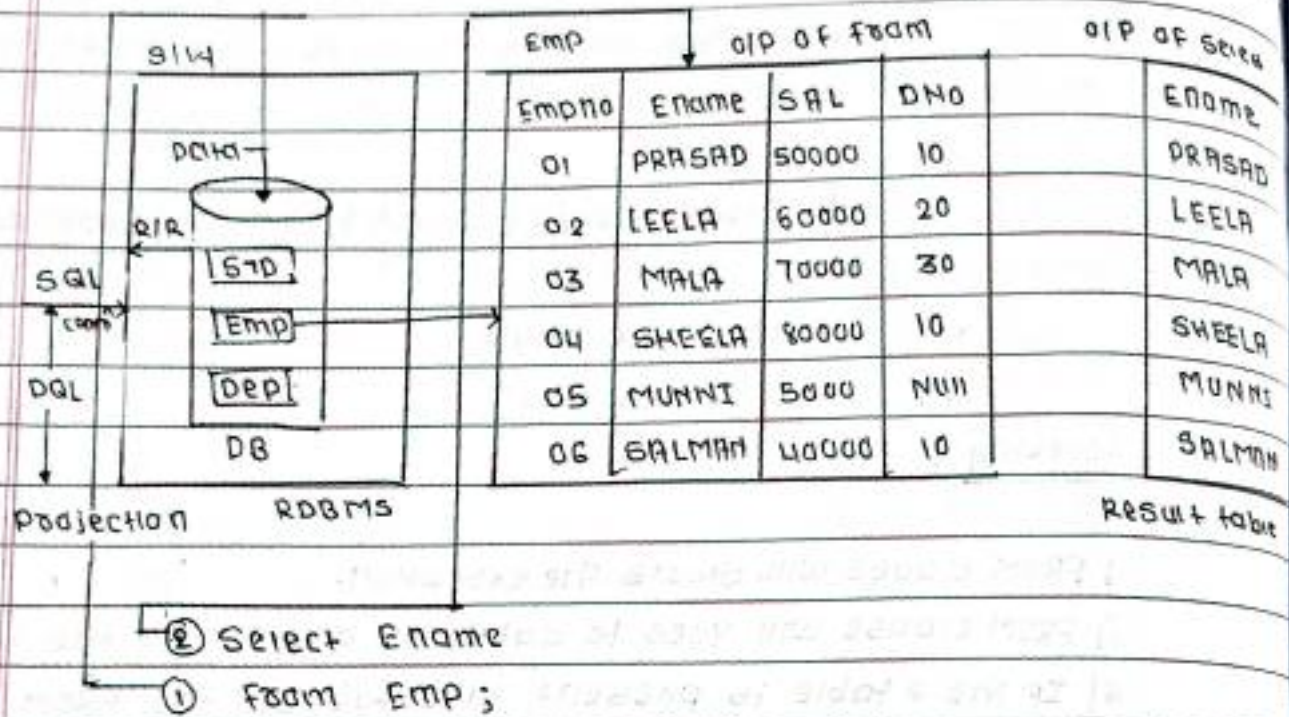
col name

Working :-

- 1] FROM clause will starts the execution.
- 2] FROM clause will goes to database and search the table
- 3] If the table is present in database the FROM clause will keep the table into execution area else if the table is not present the FROM clause give error message. i.e table or view doesn't exist.
- 4] After the FROM clause the SELECT clause will execute.
- 5] The SELECT clause will goes to the result of (output of) FROM clause and it will search the column in the table
- 6] If the column is present the the SELECT clause will SELECT the column and points the same column a result.
- 7] else, it will give the result as invalid identifier

Q - Write a query to display all the employee name which are present in employee table.

→ SELECT emp-name
FROM employee;



Q WAO to display a salary of all the employee which are present in employee table.

→ select ename SELECT sal
FROM employee;

Q Write a query to display employee name and the salary of all the resp. employee which are present in employee table

→ Select ENAME, SAL
FROM EMP;

Q WAO to display all the employee name and employee no. and salary of all the employee from employee.

→ SELECT empno, ename, sal
FROM employee;

Q Write a query to display all the details of all the employee from the employee table.

→ SELECT *
FROM emp;

Q Write a query to display employee name, Emp SAL, Emp Job and Emp department no. and all the employees.

→ SELECT ename, sal, job, dept no
FROM employee;

Q Write a query to display all the emp name, sal, one more time emp s name and the job from employee table.

→ Select, ename, sal, ename, job
from emp;

Q Write a query to display all the details from the of employee along with this one more time all the employee name.

→ Select *, ^{Emp.}ENAME
from emp;

• In the SELECT clause along with * (astrick) it is difficult to access directly to extra columns in order to access it we need to use.

SELECT TABLE_NAME.*, EXTRA COLUMNS
FROM TABLE_NAME;

Q Write a query to display all the details of emp along with one more time emp name and emp salary and the job.

→ SELECT EMP.*, ENAME, SAL, JOB
FROM EMP;

$$= 1100$$

$$N\% \text{ deducted sal} = \text{sal} - \text{sal} * N/100$$

Q → Write a query to display employee name, employee salary, and the annual salary of all the employee.

→ `SELECT ename, sal, sal * 12
FROM emp;`

Q → Waa to display employee name, emp actual salary, emp annual salary including the quarterterm salary of all the employee

→ `SELECT ename, sal, sal * 12, sal * 3
FROM emp;`

Q Waa to display all the details of employee including emp annual salary and emp 10% incremented salary and also display 25% decremented salary.

→ `SELECT emp.*,
FROM emp;`

Q Waa to display emp actual salary, emp annual salary including 10% increment in actual salary, 10% increment in annual salary for all the employee.

→ `SELECT sal, sal * 12, sal + sal * 10/100, (sal + sal * 10/100) * 12
FROM emp;`

Q Waa to display all the details of employee along with 25% deduction in actual sal, 10% increment in annual sal, 10% increment in halfterm sal, 10% increment in quarterterm sal for all employee.

→ `SELECT emp.*, sal - sal * 25/100, (sal + sal * 10/100) * 12, (sal + sal * 10/100) * 6,
(sal + sal * 10/100) * 3`

`FROM emp;`

Q. Write emp name, salary and joining date of all emp.
→

```
SELECT ename, sal, Hireddate, Sysdate-Hireddate,  
       (Sysdate-Hireddate) / 365  
FROM emp;
```

ALIAS

Alias is an alternative name which we can use for any column or any expressions or any table in order to print the user-friendly result.

There are three methods to assign the alias.

- 1] By using the AS keyword
- 2] Without using AS keyword
- 3] By using double quotes (" ").

In most of the cases all the SQL developers they will prefer to use double quotes procedure.

Note :- By using AS keyword and without using AS keyword this 2 methods were having some set of drawback.

1] The Alias name shouldn't start with any numbers and special characters.

2] The alias name should be only one string

3] To overcome this drawback we use the double quotes procedure to assign Alias name

Example: SELECT Ename A, Sal B, Job C, Deptno D
FROM emp;

Q- How to display all the emp names and emp salary and annual sal of all the emp from emp table, use the alias name for all the expressions.

→ SELECT emp name, sal, sal * 12, "ASALARY"
FROM emp;

Q How to display all the details from table and annual salary of all emp and also display 25% of deduction in annual salary of all the emp, use alias name for all the expressions.

→ SELECT emp name, sal, sal * 12, (sal - sal * 25 / 100) * 12 ("ASALARY")
FROM emp;

SELECT emp.*, sal * 12 "ASALARY", (sal - sal * 25 / 100) * 12 "25% Deduction Salary").

FROM emp;

Q How to display all the emp names, emp salary, and 20% increment in annual sal and also display the total number of years of experience for all the emp.

→ SELECT Ename, sal, (sal + sal * 20 / 100) * 12 "20% Hike",
(SYSDATE - HIREDATE) / 365 "YEARS"

FROM emp;

Q How to display all details of emp and the exp of all emp in terms of days, in terms of months, — years.

→ SELECT emp.*, (SYSDATE - HIREDATE "DAYS", (SYSDATE - HIREDATE) / 365
"YEARS" * 12, (SYSDATE - HIREDATE) / 365 * 12 "Months"

FROM emp;

Q- How to display all the employees data by selecting the following column followed by the alias name of each column

→

ENAME	- "payal"
Tob	- "Snehal"
Sal	- "Hoshu"
Dept no	- "Riya"
HIREDATE	- "preet"

DISTINCT :

It is a keyword which is used to remove the duplicated records from the result table.

- 1] Distinct and unique both are completely different.
- 2] While using the distinct keyword in SELECT clause distinct should be the first argument
- 3] In the SELECT clause we can use only once the distinct keyword.
- 4] The distinct we can also apply for asterisk (*)

Q How to display all the job from emp table but there should not be any repeated records in result.

→

```
SELECT DISTINCT job
FROM emp;
```

Q. WQTD to display the job and dept no from emp table but there should not be any repeated records.

→ SELECT DISTINCT Job, dept no
FROM emp;

Assignment

1] WQTD Emp name from emp table?

→ SELECT ename FROM emp;

2] WQTD salary and employees name from employee table?

→ SELECT sal, emp name FROM emp;

3] WQTD name, dept no and job that are present in emp table

→ SELECT ename, deptno, job FROM emp;

4] WQTD HIREDATE and deptno for all employee

→ SELECT hiredate, deptno FROM emp;

5] WQTD mgr and empno from Employee table

→ SELECT mgr, empno FROM emp;

6] WQTD All details of all employees

→ SELECT * FROM emp;

7] WQTD All department names from Dept table

→ SELECT dname FROM dept;

38] WRITTO Job as Designation, Hiredate as joining date,
 mgr as Reporting manager Number for all the emp.
 → SELECT job AS Designation, hiredate AS "joining Date",
 mgr AS "Reporting manager Number"
 FROM EMP;

39] WRITTO Job with deptno for all the emp using alias as
 Designation for the job and dno for deptno.
 → SELECT job AS Designation, deptno AS DNO
 FROM EMP;

40] WRITTO all details of an employee along with bonus
 like in sal use ALIAS AS Bonus for all the emp.
 → SELECT emp.*, Sal * 12/100 AS Bonus
 FROM EMP;

• Selection :-

It is the process of fetching the data from table by selecting
 both columns and records

- 1] The selection method is also known as filtration process
- 2] There are 2 types of filtration process
 - 1] Record Filtration
 - 2] Group filtration.

3] Record Filtration :-

By using WHERE clause we can easily filter the records
 def: Filtering individual row (records) from table based on condition

Group Filtration :-

Filtering groups of rows after applying grouping
 clause using HAVING clause

- WHERE clause :- It is used to filter the records by using the filter conditions.

Syntax : SELECT * / col / Exp [A] FROM Table - name WHERE Filter condition
--

- The FROM clause execute first
- After the FROM clause the WHERE clause execute second.
- The WHERE clause will execute Row by Row and checks condn.
- If the condition is true that record will be selected for result else the record will be ~~resulted~~ ~~not~~ selected
- The WHERE clause will gives result in form of boolean values (true and false)
- In the WHERE clause we must have to ~~right~~ ^{write} the filter condition as argument which is used to filter record based on true or false

• Format of filter condition :- (colname / expression operator val)

① ② ③

- After the WHERE clause the SELECT clause will execute and it will print the final result.

Q- Write all the emp name for all the emp the emp who are working in dept no 20

→ SELECT Emp name
FROM Emp
WHERE DNO = 20;

EMP						O/P of WHERE		
		ENAME	SAL	DNO		ENAME	SAL	DNO
10 = 20 (F)								
20 = 20 (T)	R1	SMITH	1000	10 X		SCOTT	2000	20
30 = 20 (F)	R2	SCOTT	2000	20 ✓		MILLER	4000	20
10 = 20 (F)	R3	ALLEN	3000	30 X				
Null = 20 (F)	R4	ADAMS	5000	10 X				
20 = 20 (T)	R5	KING	7000	Null X				
Execution	R6	MILLER	4000	20 ✓				

Q. WAP to all the emp name the emp whose salary are less than 5000

→ SELECT ENAME
FROM EMP
WHERE SAL < 5000;

O/P of WHERE			O/P of SELECT	
ENAME	SAL	DNO		ENAME
SMITH	1000	10		SMITH
SCOTT	2000	20		SCOTT
ALLEN	3000	30		ALLEN
MILLER	4000	20		MILLER

Q. WAP to all the details of emp the emp who are working as Salesman

→ SELECT emp.*
FROM emp
WHERE job = 'Salesman'; → due to string data.

It should be in uppercase as shown in table
'SALESMAN';

Note :

- 1] The SQL is a not case sensitive language
- 2] but the data is case sensitive
- 3] In SQL the string data and as well as the data related data we has to write in single quotes (' ').

Q. WAQTD all the details of emp, the emp who hired on 28 sep 1981

→ SELECT *
FROM EMP
WHERE HIREDATE = '28-SEP-1981';

Q. WAQTD all the details of emp, the emp who hired after 1981

→ SELECT *
FROM EMP
WHERE HIREDATE > '31-DEC-1981';
OR
>= '01-JAN-1982';

Q. WAQTD all the details of emp along with the annual sal of emp the emp whose annual sal is more than 30,000.

→ SELECT EMP.*, SAL*12 AS salary
FROM EMP
WHERE SAL*12 > 30000; OR ^{Alias name.} WHERE salary > 30000; ↑
WHERE

Q. WAQTD The Alias name from the SELECT clause it is difficult to access as a filter condition bcz of execution process

① FROM → ② WHERE → ③ SELECT

If you want to use the ALIAS name in WHERE clause use need to use inline subquery

Q. WRTD all details of employees except all the emp who are working in dept no 10.

→ SELECT *
FROM EMP
WHERE deptno != 10 ;

Q. WRTD all details of employees except the emp who are working in dept no 10 and dept no 20

→ SELECT *
FROM EMP
WHERE deptno != 10 AND deptno != 20 ;

Logical operators :-

The logical operators are used to pass the multiple filter condition to the WHERE clause and HAVING clause

These are the logical operators in SQL :-

- 1] AND operator or we called it as (Logical multiplication gate)
- 2] OR (Logical addition operator)
- 3] NOT (Inverse operator)

(*) AND $\Rightarrow$

(+) OR $\Rightarrow$

NOT $\Rightarrow$

O/P1	O/P2	Result	O/P1	O/P2	Result	O/P	Result
1	0	0	1	0	1	0	1
0	1	0	0	1	1	1	0
0	0	0	1	1	1		
1	1	1	0	0	0		

Q- Display all the details of emp, except all the emp, the emp who are working in dept no 30.

→
SELECT *
FROM emp
WHERE NOT DEPTNO = 30;

SPECIAL OPERATORS :-

These operators are used to perform special task in SQL.

There are several type of special operators in SQL :-

- 1] IN VS NOT IN
- 2] BETWEEN VS NOT BETWEEN
- 3] IS VS IS NOT
- 4] LIKE VS NOT LIKE
- 5] ESCAPE CHARACTER
- 6] REGULAR EXP (REGEXP)

1] IN VS NOT IN

This are the special operators in SQL which are used to handle multiple values at the RHS side for comparison.

IN operator :- It is used to handle the multiple values at the RHS side for the comparison and it will gives the result as true if any one value from RHS will satisfies the condition.

NOT IN operator :- It is totally opposite to IN operator.

Syntax :- WHERE COL-NAME / EXP IN / NOT IN (V1, V2, V3, ... Vn);

Q- Write a query to find all the emp details, including annual sal and the experience of all the emp in terms of years for all the emp who are working in dept no (10, 20, 30) and earning sal more than 800 and hired before 1988 except SALESMAN, manager and president.

→ SELECT * Emp.*, sal * 12 "ANNUAL SALARY", (sysdate - hiredate) / 365 "EXP IN YEARS"
FROM EMP
WHERE sal > 800 deptno IN (10, 20, 30) AND sal > 800 AND
hiredate <= '31-Dec-1987' AND job NOT IN
('SALESMAN', 'MANAGER', 'PRESIDENT');

BETWEEN VS NOT BETWEEN :

This operators are used to solve the range related queries in SQL.

Syntax :- WHERE col_name / EXP BETWEEN lower_range AND higher_range;

Example :-

Q- Write a query to find all the emp whose sal are betⁿ 1000 to 3000.

→ SELECT *
FROM EMP
WHERE sal BETWEEN 1000 AND 3000 ; / 1001 AND 2999 ;

Q- Write a query to find all the emp whose sal in range of 1500-3000.

→ SELECT *
FROM EMP
WHERE

Q. Write all details of emp, excluding all details of emp, the emp whose sal in range of 1600 to 5000

→ SELECT *
FROM EMP
WHERE SAL NOT BETWEEN 1600 AND 5000;

• LIKE VS NOT LIKE :-

• This operators are used to perform the pattern matching operation

Syntax :- WHERE COL / EXP LIKE / NOT LIKE 'PATTERN';

Q. • To create the patterns in SQL we need to use 2 special characters (Wild card character)

- 1] % → allow any type of char / n-no. of char
- 2] _ (underscore) → It will also allow . . . , but allow only 1 digit char

• properties of modulus (%) :-

- 1] It will allow any types of characters
- 2] It will allow n no. of characters
- 3] It will also allow % and _ .

• properties of underscore (_) :-

- 1] It will allow any type of character
- 2] It will allow only one digit of character
- 3] It will also allow % and _ .

Regular Expression :-

It is the updated version of LIKE which is used to perform the pattern matching operations.

Syntax : WHERE REGEXP_LIKE (COL/EXP, 'PATTERN', 'C1');

In the regular Expression we need to create pattern we use different type of wildcard character those are.

- 1) (cap) → ^ → Starting with
- 2) (dollar) → \$ → Ends with
- 3) → . → SINGLE character like.
- 4) → * → Multiple character
- 5) (array) → [] → To present the homogeneous data / char
- 6) (hyphen) → - → Range type character

Note :- i indicates that case insensitivity

Q. Write all the details of employee, the emp whose name ends with consonants.

→

```
SELECT *
FROM EMP
WHERE NOT REGEXP_LIKE (ename, '.*[a,e,i,o,u,s]$', 'i');
```

Q. Display all the details of emp, the emp whose name having second character as A.

→

```
SELECT *
FROM EMP
WHERE REGEXP_LIKE (ename, '^A.*');
```

Q. WROTE all details of emp, the emp whose name starts with ^.

→ SELECT *

FROM EMP

WHERE REGEXP(ename, '^%') ENAME LIKE '^%';
- like

ESCAPE CHARACTERS :-

The characters which are used to remove all the properties of all the wildcard characters and make them as normal char.

Q. WROTE all the details of emp, the emp whose name starts with % (without using REGEXP).

→ SELECT *

FROM EMP

WHERE ename LIKE '\%.\%'\ESCAPE '\';

Q. Display all details of emp, the emp whose name starts with % and ends with _ (without using REGEXP).

→ SELECT *

FROM EMP

WHERE ENAME LIKE '1%1%' ESCAPE '1';
↑ underscore and not

• IS Vs IS NOT operator

This operators are used to compare null values.

Syntax:- WHERE col / EXP IS / IS NOT NULL;

↳ not a zero

• NULL is not a zero

• NULL = NULL (false)

• NULL is Empty cell

• Any arithmetical operation which we are going to perform

⊗ Null values it will gives result as Null.

• We cant do any mathematical values on Null values.

① → To find all the details of emp, the emp whose commissions are null

SELECT * FROM Emp
WHERE comm IS NULL;

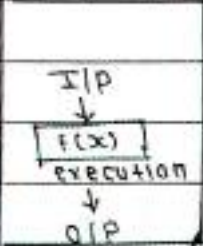
→, The emp whose

① → display all the details of emp, the emp who are not having any reporting managers.

WHERE MGR IS NULL;

FUNCTIONS

Defination - The set of instructions which are used to perform an any specific task or an operation are known as functions.

	The major component of any functions are : 1] Input (parameter) 2] execution plan / flow / Working procedure 3] output.
---	--

The functions has been classified into 2 types :-

1] user defined function (UDF).

2] Inbuilt or builtin function.

- It is static in nature
- all the users can access easily.

1] user defined function :

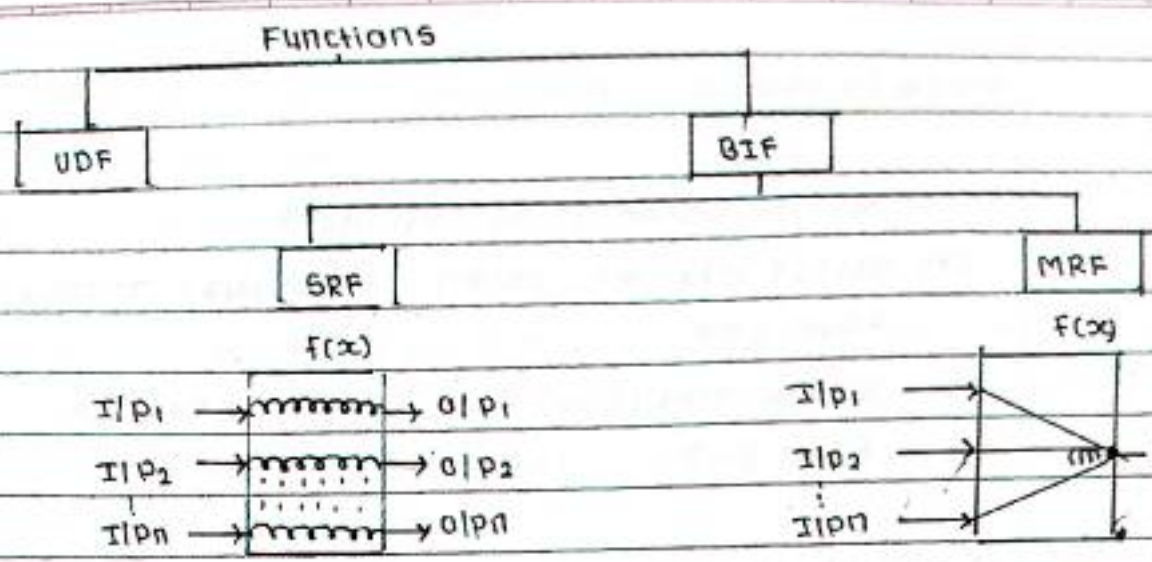
Function which are created by the users based on requirements.

- The user defined funⁿ are also known as dynamic funⁿ.
- Only the limited users can able to access the UDF.
- This function we can easily modify according to requirements.

2] Builtin function :

The functions which are predefined in each and every computer are known as builtin function.

- The Builtin funⁿ also known as static funⁿ.
- This funⁿ all the users can access
- modification is not possible in built funⁿ.
- The builtin funⁿ has been classified into two types.
 - 1] single row funⁿ (SRF) 2] Multi row funⁿ (MRF)



The funⁿ which will allow n no of
i/p and executes, finally which
will give the n no of o/p with
to i/p are known as single
row function

- Single row funⁿ will execute
row by row
- There is no aggregation point
- The no of input = no of output
- Ex :-
1] length function
2] Sub-string ()
3] Length () - (upper(), lower())
4] Initcap ()
5] upper ()
6] Lower ()

The funⁿ which allow n no.
of i/p and aggregate in
single place and executes and
finally which gives only 1
o/p this type of funⁿ is called
as Multirow funⁿ.

- Multi row funⁿ will exe
group by group
- MRF also known as aggre-
gate funⁿ and group funⁿ
- In MRF there will be a
aggregation point
- Example :-
1] max () - (use numbe, date)
2] min ()
3] Avg ()
4] sum ()
5] count ()

RULES OF MULTICROW FUNCTIONS :

1] MRF will allow only one argument

EX:- SELECT MAX (SAL, COMM)
FROM EMP;

ERROR :- Invalid no. of argument

EX:- SELECT MAX (SAL)
FROM EMP;

It's correct

2] In the SELECT clause we can use n no. of multicrow functions

EX:- SELECT MAX (SAL), MIN (SAL), AVG (SAL), SUM (SAL), COUNT (SAL),
MAX (HIREDATE), MAX (ENAME)
FROM EMP;

3] In the WHERE clause it is not possible to access the MRF

EX:- SELECT * FROM EMP

WHERE SAL = MAX (SAL); → error :- group Funⁿ are not allowed

Reason :-

The execution process of WHERE clause and MRF are totally different [correct one → WHERE SAL = (SELECT MAX (SAL) FROM EMP)

4] In the SELECT clause along with multicrow Funⁿ it is difficult to axis the any normal columns until we are not applying or using the grouping concept.

Reason :- The result will be not in form of table format after applying the grouping concept it will create the result in the table format.

EX: SELECT COUNT (EMPNO), DEPTNO
FROM EMP

Group by deptno ;

5] The MRF will ignores the null values

EX: SELECT COUNT (SAL)

FROM EMP ;

→ 1101P = 14 ✓

6] The among all the MRF the count is only one MRF which will be applicable for all the datatypes and in the count function we can use asterisk (*) as a argument.

EX: SELECT COUNT (*)

FROM EMP ; → only primary value ans will come ✓

• Null value is not allowed in primary key.

• Number of recorded count all the record

if primary key isn't there

7] We can also use the nested funⁿ (funⁿ inside an another funⁿ) is known as nested functions.

EX:- SELECT MAX (COUNT (*))

FROM EMP

GROUP BY DEPTNO ; ✓

Q] What is the max sal from emp table

SELECT MAX (SAL) OR SELECT MAX (SAL) "MAXIMUM SAL";

FROM EMP ;

Q] Display total sal for all emp.

SELECT SUM (SAL)

FROM EMP ;

Q] Display total sal which is needed to pay for all emp working as sales

→ SELECT SUM (SAL)

FROM EMP WHERE JOB = 'SALESMAN';

GROUP BY CLAUSE

It is used to convert a records into multiple groups.

Syntax :-
SELECT MRF / Group Expⁿ
FROM Table name
WHERE filter condition → optional
GROUP BY columns;

Imp Note : We can't use asterisk (*) if there is group by clause.

Execution process :-

- FROM clause will execute 1st.
- If we are using WHERE clause it execute 2nd else the group by clause will execute 2nd.
- The group by clause will execute row by row.
- Once the group by clause will execute the records from the table will convert as a multiple groups.
- After the group by clause finally SELECT clause execute and it will print final result.

- The clauses which will exe after groupby clause are going to use Group By Group.
- The column name which we are using in groupby clause we can also use same col. in SELECT clause those all the column are known as group expressions.

Q. Know the no. of emp who are working in each and every dept which are present in emp table.

→ SELECT COUNT(*) //, deptno;
FROM emp
Group by Deptno;

we can also write this col here too.

O/P of FROM

		Ename	DNO	SAL	Job
③ SELECT COUNT(*)	→ R1	Smith	10	1000	S
① FROM emp	→ R2	Scott	20	2000	M
② Group by DNO;	→ R3	King	10	1000	P
	→ R4	Allen	30	5000	S
	→ R5	Admas	10	6000	P
	→ R6	Miller	30	7000	S

O/P of Groupby = 4

G1 DNO = 10

Ename	DNO	SAL	Job
Smith	10	1000	S
King	10	1000	P
Admas	10	6000	P

Count(*)	DNO
3	10
1	20
2	30

G2 DNO = 20

Scott	20	2000	M
-------	----	------	---

G3 DNO = 30

Allen	30	5000	S
Miller	30	7000	S

O/P of SELECT

Having Clause.

Having clause is used to filter the groups which has been created by Groupby clause.

Syntax :- SELECT MRF/Group
FROM Table name
WHERE Filter condition
Group by column
Having Group filter Condition ;

Working :-

- 1] First the FROM clause execute
- 2] If we are using WHERE clause, the WHERE clause execute 2nd (if present) else the compiler goes for Groupby clause is executed
- 3] Groupby clause executes row-by-row converting the records into 'n' no of groups.
- 4] After the Groupby clause the having clause will execute
- 5] The HAVING clause will execute group by group
- 6] For having clause we need to pass the group filter condition as argument, if the condition is true for that group it is selected for result else the group is rejected
- 7] Finally SELECT is executed and displays the final result.

Note : To execute having clause, Groupby clause is mandatory.

Q. WRTD, the deptno in which exactly two emp's are working

→ SELECT count(*), deptno
FROM emp

Group by deptno

HAVING count(*) = 2;

↓ EMP

ENAME	SAL	DEPTNO
SMITH	1000	10
SCOTT	2000	20
KING	3000	10
ALLEN	5000	20
ADAMS	7000	30
MILLER	8000	10

G1 DEPTNO = 10

SMITH	1000	10
KING	3000	10
MILLER	8000	10

count(*) = 3

3 ≠ 2 (F)

G2 DEPTNO = 20

SCOTT	2000	20
ALLEN	5000	20

O/P OF

G2 DEPTNO = 20

SCOTT	2000	20
ALLEN	5000	20

2 = 2 (T)

count(*) deptno

2 20

G3 DEPTNO = 30

1 = 2 (F)

ADAMS 7000 30

Q. WRTD, the Job from emp table in which the total salary which is needed to pay more than 8000.

→ SELECT SUM(SAL), Job
FROM emp

Group by Job

HAVING SUM(SAL) > 8000;

Q. WRTD, the deptno in which the total no-of emp's are more than 3 and less than 6 emp's.

→ SELECT count(*), deptno
FROM emp

Group by deptno

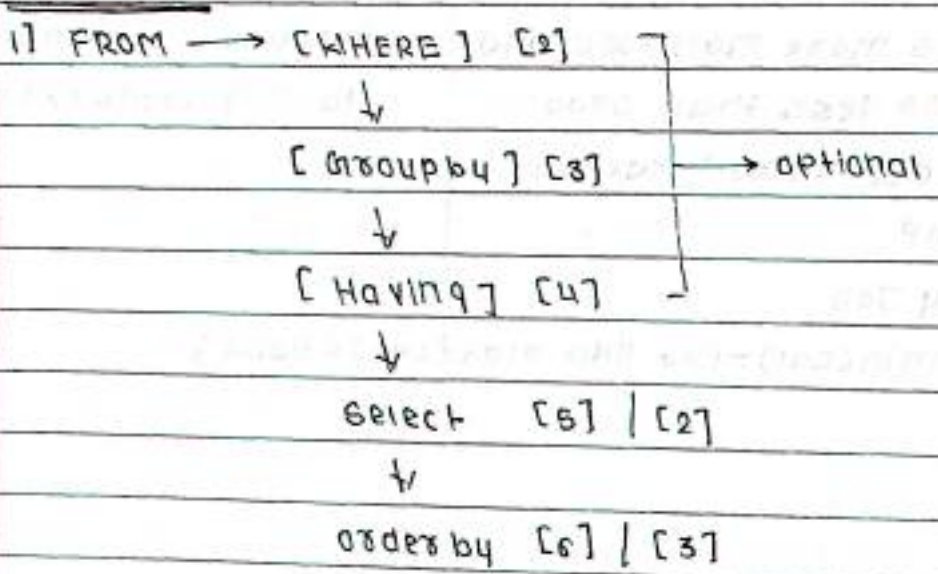
HAVING count(*) > 3 AND count(*) < 6;

ORDER BY CLAUSE

This clause is used to arrange the result either in ascending order or in descending order.

Syntax : `SELECT COL/EXP/ *
FROM Table name
[WHERE FILTER CONDITION] → optional
[Groupby columns] → optional
[Having group filter condition] → optional
ORDER BY COL-NAME [ASC] / DESC;`

Working :-



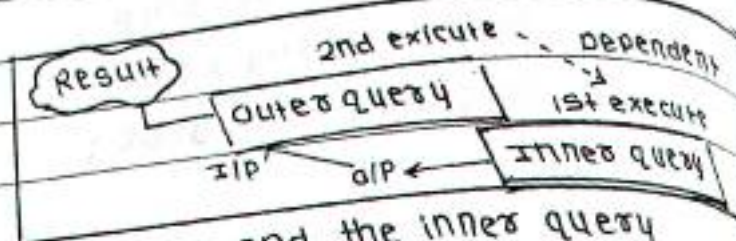
2] In the order by clause we can also use the multiple columns, if the data is present according to req. It will look for all the multiple col else the order by clause will work only for the first column.

3] The order by clause will executed after the select clause
 4] Since, the order by clause will execute after the select clause, the alias name from select we can easily access in order by clause.

Sub-query

The query written inside another query is known as subquery.

Working :-



- 1) Let us consider 2 queries outer query and the inner query
- Inner query will execute first
- The inner query will return the partial output
- The o/p of inner query will act as input for the outer query
- After the inner query the outer query will execute second and finally it will give the final result.
- In the subquery always the outer query will depend on inner query
- We can able nest upto 255 subqueries
- We can use the subquery concept in select clause, in from clause, in where clause also in having clause.

Sub-query for WHERE CLAUSE

Why subquery in the where clause?

Reason :-

- 1) Whenever the unknown values are present in the question we need to use the sub-query
 - 2) There are 2 cases for sub-query
- case-1) The data to be selected
- 2) Condⁿ to be executed.
- both are from same table.

case 2] the data to be selected

The condⁿ must b to be executed
both are from different table,

Case 1 Example :-

① WROTD all the emp names whose sal's are more than 'KING'.

→ SELECT ENAME

FROM EMP

WHERE sal > (select sal
 ^{int} FROM emp
 WHERE ENAME = 'king');

↓
o/p => Adams

EMP

DEPT

ENAME	SAL	DNO	DNO	DNAME	LOC	
Smith	1000	10	10	HR	Pune	1000 > 7000 (F)
Scott	8000	20	20	Sales	Hyd	8000 > 7000 (F)
King	7000	Null	30	Prod	KAR	
Adams	9000	10				
Aliens	6000	20				

① WROTD all the emp name the emp who is working in same deptno of Scott.

→ SELECT ENAME

FROM EMP

WHERE Job Deptno = (select Deptno
 FROM emp

WHERE ENAME = 'scott');

Types of Subquery

Based on the result of inner query the subquery has been classified into two types.

- 1] Single row subquery (SRS)
- 2] Multi row subquery (MRS)

1] Single row subquery.

If the inner query returns exactly one value as a output this types of subqueries are known as single row subqueries.

2] Multi Row Subquery (MRS) :-

If the inner query returns more than one value as o/p this types of subquery are known as MRS.

Q → WROTD the loc of all the emp whose name starts with k. (SRsq)
 SELECT LOC FROM DEPT WHERE DNO = (SELECT DNO FROM EMP
 WHERE ENAME LIKE 'K%');

DNO = 10 ✓ → Single o/p
 20 (SRs)
 30

LOC = Pune

EMP			DEPT			
ENAME	SAL	DNO	DNO	DNAME	LOC	
Smith	1000	10	10	HR	Pune	
Scott	2000	20	20	SALES	Hyd	
King	7000	10	30	PROD	KAR	
Allen	1000	20				
Admas	2000	10				

Q → Display the loc of emp whose name starts with A. (MRSq)
 SELECT LOC FROM DEPT WHERE DNO = (SELECT DNO FROM EMP
 WHERE ENAME LIKE 'A%');

Note :-

By analyzing the ques it is very difficult to identify the type of sub-query (SRsq) or (MRsq) to avoid confusion we use special operators (IN or NOT IN) and we even use subquery operators are (ALL and ANY) We can also use MR Functions

ALL and any

These are the subquery operators used to deal with multiple o/p from the inner query in case of relational comparison ($>$ & $<$).

- 1] The ALL operator gives true if all values from RHS becomes true, while making relations.
- 2] The any operator will gives the result as true if any one value will satisfied the condⁿ while making the relation.
- 3] The ALL operator will work similar to AND op. and any op will work similar to OR operator.

Ex:-

- 1] WROTD All the details of emp, the emp whose sal are more than all emp of deptno=10.

→ `SELECT * FROM EMP WHERE sal > ALL (SELECT sal FROM EMP WHERE deptno = 10);`

O/P → ALLEN 7000 20 (3000, 4000, 6000)

- 2] Display all the detail of emp, the emp whose sal are more than any one of the emp of dept=10.

→ `SELECT * FROM EMP WHERE sal > ANY (SELECT sal FROM EMP WHERE deptno=10);`

EXPLAN: $3000 > 3000 \times$ $5000 > 3000 \checkmark$
 $4000 \times$ $4000 \checkmark$
 $6000 \times$ $6000 \times$

EMP			
ENAME	SAL	DNO	
SMITH	3000	10	
SCOTT	5000	20	
KING	6000	10	
ALLEN	7000	20	
ADAMS	4000	10	
MILLER	2000	30	

Relationship Between the columns (Hierarchy betn columns)

In this concept we need to deal with the higher hierarchy and we need to deal with lower hierarchy.

Example:-

EMP

PID DNAME LOC

1] In emp table → Empno MGR

2] In student table → Std Lead id

3] In player table → PID captain id

Examples on emp table :-

- In the emp table we can get the higher hierarchy as finding managers and lower hierarchy as finding emp.
- If we are dealing with the higher hierarchy chances are there we will get either the higher hierarchy data or the lower hierarchy data.
- If we are dealing with the lower hierarchy data chances are there we will get either we will get lower hierarchy data or higher hierarchy data.

Q- WHAT is the Smith manager name

SELECT Empname FROM emp

WHERE MGR = (SELECT Empno IN (SELECT MGR FROM emp
WHERE Empname = 'Smith'));

EMP

DEPT

Empno	Ename	MGR	Dno	dno	dname	LOC
01	Smith	05	10	10	HR	Hyd
02	Scott	03	20	20	Sales	Pune
03	King	Null	10	30	Prod	KAR
04	Allen	03	Null			
05	Admas	06	10			
06	Millars	01	20			

Q- WHAT is Millars MGR name

SELECT Empname FROM emp

WHERE Empno IN (SELECT MGR FROM emp

WHERE Ename = 'Millars'); o/p → Smith

Q- WHAT are all the emp name the emp who are reporting to King

SELECT Ename FROM emp

WHERE MGR = IN (SELECT Empno FROM emp

WHERE Empname = 'King'); o/p → Scott
Allen

Q- WHAT is Smith's manager's MGR name.

SELECT Empname FROM emp

WHERE Empno IN (SELECT Empno FROM emp

WHERE Ename = Empno IN (SELECT MGR
FROM emp

o/p: - Millars

WHERE Ename
('Smith'));

Subquery in Having clause.

We can use the subqueries in the having clause if there is any unknown value are present in question while filtering the group.

Q. WQOTD the deptno in which exactly five emp are working

```
SELECT count(*), deptno  
FROM emp  
group by deptno  
having count(*) = 5;
```

Q. WQOTD the deptno in which max no-of emp's are working

```
SELECT deptno, count(*)  
FROM emp  
group by deptno  
having count(*) in (select max(count(*)) FROM emp  
group by deptno);
```


Q- WRTD the deptno in which the total sal which is needed to pay is more than the total salary of all the job

→ SELECT deptno
FROM emp

GROUP BY deptno

HAVING SUM(sal) > (SELECT SUM(sal) FROM emp

WHERE Job is not NULL);

GROUP BY Job);

Q- WRTD the deptno in which the max sal is needed to pay is more than the total sal of any one job

→ SELECT deptno
FROM emp

GROUP BY deptno

HAVING MAX(sal) > ANY (SELECT SUM(sal) FROM emp

GROUP BY Job);

SUBQUERY IN FROM CLAUSE (Inline Subquery)

To access the alias name from SELECT clause to WHERE clause as filter condn we need to use subquery (inline) from the FROM clause.

Q- WRTD all the details of emp the emp whose annual sal is more than 30000

→ SELECT *

FROM (SELECT Emp.*, SAL*12 AS salary FROM emp)

WHERE ASsalary > 30000;

Subquery in SELECT clause.

In order to access the general col's with multirow funⁿ without using Group clause we need to use subqueries in SELECT clause.

Q- Write to all the emp's name and the total sal of complete emp table.

→ SELECT ENAME, (SELECT SUM(sal) FROM emp)
FROM emp;

Q- Write to all the Jobs from the emp table along with the total sal which is needed to pay for deptno 30.

→ SELECT deptno

SQL deals with upto 255 tables only (drawback)
We will deal with the multiple table in subquery too but,
In subquery we can't fetch data from both table.

Page No.	
Date	

JOINS

- It is the process of fetching the data from all the multiple tables simultaneously is known as Joins.

- Joins are faster than subquery.

- In SQL the joins we can also call it as merge.

- There are 5 diff type of joins in SQL:-

- 1] Cartesian join / cross join

- 2] Inner join / equi join

- 3] Natural join

- 4] Self join

- 5] Outer join + 1] Left outer join

- 2] Right outer join

- 3] Full outer join.

- While joining any multiple tables, all the multiple table get merged and it will form one bigger table and that table consider as result table else, whatever the bigger table we got the main result will be selected from the bigger table itself.

- The joins has been classified into 5 types that classification is done based on types of result which we are going to take in result table.

- Whenever we are joining multiple table into result table we can able to get the 3 types of records

- 1] Error record

- 2] Matching record

- 3] Unmatching record.

17 cartesian join

It is the process of fetching the data from all the multiple table based on the crossing principle.

Crossing Principle:

each and every record in first table are going to be merged with each and every records of all the remaining table is known as crossing principle.

Cartesian Syntax:-	SELECT col / exp / *
(Oracle Syntax)	FROM TABLE1, TABLE2, --- TABLEN
	[WHERE FILTER CONDITIONS] --- OPTIONAL

(ANSI)	SELECT col / exp / *
American national	FROM TABLE1, TABLE2, - cross join
Standard institute	TABLE2 cross join
	TABLEN
	[WHERE FILTER condition] → optional

Q - Write all the husband names and the wife name which are present in both Husband and wife table.

→ SELECT Hname, Wname
FROM HUS, WIFE;

HUS			WIFE	
HID	Hname	WID	WID	Wname
H1	PHONI	W2	W1	Anushka
H2	HARDIK	NULL	W2	Sakshi
H3	SALMAN	NULL	W3	Alia
H4	VIRAT	W1		

OIP OF FROM CLAUSE

HID	Hname	WID	WID	Lname
H1	DHONI	W2	W1	ANUSHKA
H1	— " —	W2	W2	SAKSHI
H1	— " —	W2	W3	ALIA
H2	HARDIK	N	W1	ANUSHKA
H2	— " —	N	W2	SAKSHI
H2	— " —	N	W3	ALIA
H3	SALMAN	N	W1	ANUSHKA
H3	— " —	N	W2	SAKSHI
H3	— " —	N	W3	ALIA
H4	VIRAT	W1	W1	ANUSHKA
H4	— " —	W1	W2	SAKSHI
H4	— " —	W1	W3	ALIA

Hname	Lname
DHONI	ANUSHKA
DHONI	SAKSHI
DHONI	ALIA
HARDIK	ANU...
HARDIK	SAK...
HARDIK	ALIA
SALMAN	ANU...
SALMAN	SAK...
SALMAN	ALIA
VIRAT	ANU...
VIRAT	SAK...
VIRAT	ALIA

OIP OF SELECT CLAUSE.

Cartesian join drawback: Too many records (don't use it)
to overcome this we use inner join
(key attributes):- Imp to make connection betⁿ 2 table (pk, fk)

Page No.

Date

Note :-

- The Cartesian join is also known as cross join or the traditional method
- The Cartesian join will print completely the too many records this will be the drawback of Cartesian joining to overcome from this drawback we must use inner join

Q - How to Ename and bname and Job, loc present in emp table for all the emp

→ `SELECT bname, ename, job, loc
FROM dept, emp;`

Inner Join :-

Inner join is used to get the matching records from the multiple tables.

1] If the table are having connection with the help of FK then only its possible to get matching record from multiple table.

2] To get matching record from multiple table we must have to use the join condⁿ. (The join condⁿ is nothing but the connectivity betⁿ multiple table by using pk and fk).

3] Syntax :-

Syntax of inner join :-
(example)

↓
use this one
better than Ans 1

```
SELECT COL/EXP/*  
FROM TABLE1, TABLE2, --- TABLEN  
WHERE INNER JOIN CONDITION OPERATOR  
FILTER CONDITIONS
```


Syntax for inner join :-
 (ANSI)

SELECT col/exp/*
 FROM TABLE1 INNER JOIN TABLE2 INNER JOIN
 TABLEN
 ON INNER JOIN CONDITION
 [WHERE FILTER CONDITIONS];

Format :-

Table_name1.col = Table_name2.col
 Emp.DNO IN Dept.DNO

By using this format we remove the drawback of c1.

- Q. Write all the emp names and the emp's dept name for all the emp who are present in emp table.

→ SELECT Ename, Dname — (3)
 FROM emp, Dept — (1)
 WHERE Emp.DNO = Dept.DNO — (2)

10 = 10

20 x
 30 x → should be match

KA (key attribute)

EMP		FK	PK		Dept
Ename	DNO		DNO	Dname	
SMITH	10	→	10	HR	
SCOTT	20	→	20	Sales	
KING	Null		30	Prod	
ALLEN	20	→			

Ename	DNO	DNO	Dname		Ename	Dname
SMITH	10	10	HR		SMITH	HR
SCOTT	20	20	Sales		SCOTT	SALES
ALLEN	20	20	Sales		ALLEN	SALES

O/P of FROM

O/P of SELECT

Natural Joins :-

NJ is the combination of inner join and cartesian join.

The natural join is having two properties :-

- 1] The result of Natural join will be similar to the cartesian join if there is no common column between all the multiple tables.
- 2] The result of Natural join will be similar to the inner join if there is a common column between all the multiple tables.

The natural join act as the cartesian join or the inner join.

Syntax :- (ANSI)
 SELECT COL / EXP / *
 FROM TABLE 1 Natural join TABLE 2 Natural join Table n
 [WHERE FILTER CONDITIONS] → optional.

Q. Write a query to all the student data and dept data for all the student.

→ SELECT *
 FROM Student Natural Join Dept.

Dept

DNO	pname
10	HR
20	SALES

SID

SID	sname
S1	KARAN
S2	ARTUNE

DNO	pname	SID	sname
10	HR	S1	KARAN
10	HR	S2	ARTUNE
20	SALES	S1	KARAN
20	SALES	S2	ARTUNE

Q Display all the details of emp including emp location from all emp the emp who are working as managers.

→ ~~SELECT * FROM emp~~ IN emp.*, Loc
FROM emp NATURAL JOIN dept
WHERE Job = 'MANAGER';

↓
Error :- Column used in Natural join cannot have qualified.
Solve using Inner join.

SELECT emp.*, Loc
FROM emp, dept
WHERE emp.deptno IN dept.deptno
AND Job IN 'MANAGER';

Note :

1] The advantage of natural join :-

- We can able to use it as the cartesian join and inner join
- No need to mention any join condition
- There is column confusions.

2] The drawback of natural join :-

- It is not possible to access any table complete data with any extra columns for other tables.

Ex: SELECT emp., Loc
FROM emp NATURAL JOIN dept

Outer Join

It is used to get the matching records from both the tables and unmatched records from the respective table.

mainly there are 3 types of outer joins :-

1] Right outer join

2] Left outer join

3] Full outer join

1] Right outer join :-

Right outer join is used to get the matching records from both the tables including the unmatched records only from the right table.

2] Left outer join :-

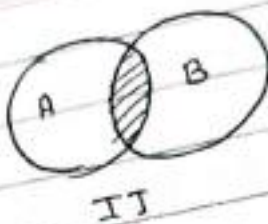
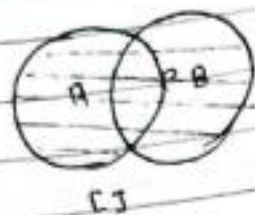
It is used to get the matching records from both table including the unmatched records only from the left table.

3] Full outer join :-

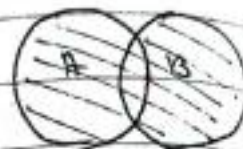
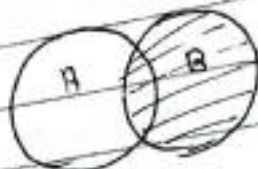
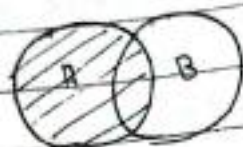
It is used to get the matching records from both the tables including the unmatched records from both the tables. (It is like cartesian)

Note :- (but the difference of cartesian)

By using outer join we can also able to get only the unmatched records from the respective table.



NJ



LOJ

ROJ

FOJ

Left outer Join

It is the process of getting the matching records from both table along with the unmatched record of left table.

Syntax :-
 (Oracle)
 SELECT COL/EXP/*
 FROM TABLE1 [LT], TABLE2 [RT]
 WHERE TABLE1 [LT].COL IN TABLE2 [RT].COL (+)
 OPERATOR FILTER CONDITIONS;

(ANSI) :-
 SELECT COL/EXP/*
 FROM TABLE1 [LT] LEFT JOIN TABLE2 [RT]
 ON LT.COL IN RT.COL
 [WHERE FILTER CONDITION]

EMP

Dept

LOJ (SELECT)

ENAME	DNO
SMITH	10
SCOTT	NULL
KING	20
ALLEN	NULL

LT

DNO	DNAME
10	HR
20	SALES
30	PRODU
40	ACC

RT

ENAME	DNO	DNAME
SMITH	10	HR
KING	20	SALES
SCOTT	NULL	NULL
ALLEN	NULL	NULL

Q. Write the emp details and their dept detail even though emp who are not working in any dept.

SELECT *	ANSI :- SELECT *
FROM Emp LT, Dept RT	FROM Emp LT LEFT JOIN Dept RT
WHERE LT.DNO = RT.DNO (+)	ON LT.DEPNO IN RT.DEPNO.

Right outer JOIN :-

It is the process of getting / fetching the data / record from both table along with unmatched records only from the right table.

Syntax :-
(ORACLE)

```

SELECT COL | EXP | *
FROM TABLE1 [LT], TABLE2 [RT]
WHERE LT.COL (+) IN RT.COL OPERATOR
FILTER CONDITIONS.
    
```

(ANSI) :-

```

SELECT COL | EXP | *
FROM TABLE1 [LT] RIGHT JOIN TABLE 2 [RT]
ON LT.COL IN RT.COL
[WHERE FILTER CONDITIONS];
    
```

Q. Write all the emp detail along with their dept details for all the dept even though in which there is no emp's are working.

SELECT *
FROM emp LT, dept RT
WHERE LT.deptno (+) IN RT.dno;

(ANSI)
SELECT *
FROM emp LT RIGHT JOIN dept RT
ON LT.DNO IN RT.DNO;

⇒	ENAME	DNO	DNO	DNAME
	SMITH	10	10	HR
	KING	20	20	SALES
	NULL	NULL	30	PRO
	NULL	NULL	40	ACC

Full outer Join :-

It is process of fetching both matching and unmatched records from both the tables

Syntax (ANSI) :-

```
SELECT COL | EXP | *  
FROM TABLE 1 [LT] FULL JOIN TABLE 2 [RT]  
ON LT.COL IN RT.COL  
[WHERE FILTER CONDITIONS];
```

①- Want to all emp data including their dept detail even though the emp who are not working in any dept, even though in the dept there is no emp's

→ SELECT *

FROM EMP LT FULL JOIN DEPT RT

ON LT.DNO IN RT.DNO

ORDER BY LT.DNO;

⇒	Ename	Dno	Dno	Dname
	SMITH	10	10	HR
	KING	20	20	SALES
	SCOTT	NULL	NULL	NULL
	ALLEN	NULL	NULL	NULL
	NULL	NULL	30	PRO
	NULL	NULL	40	ACC.

FOT

NOTE: By using outer Join we can also able to deal with only the unmatched records by adding extra condition to query

Eg:- Child table. key attribute IS NULL;

• Emp.DNO IS NULL;

Self Join :-

Joining the same table itself is known as self join.

Why Self Join :-

In order to find the lower hierarchy and higher simultaneously from same table we must have to use self join

1] The self join we can use it as the cartesian join and also inner join and also outer join.

2] In the self join since we are joining the same table we must have to use the alias name for each and every table.

To avoid the column ambiguity.

3] If we want to use the self join as a cartesian join we no need to write an any join condition.

4] If we want to use the self join as a inner join we must have to write the join condition.

This join condition is known as self join condition.

5] If we want to use the self join as a outer join, we must have to use the outer operator (+)

Self Join as a Inner Join :

To get the matching record within the same table we can use the self join.

Syntax :-

(Oracle)

```
SELECT COL/EXP/*
```

```
FROM TABLE1 ALIAS, TABLE2 ALIAS, ... TABLE N ALIAS
```

```
WHERE SELF JOIN COND OPERATOR FILTER CONDITION
```


Note:- use subquery when to find only 1 hierarchy and to use both hierarchy use self join.
Subquery disadvantage is it can't handle 2 hierarchy at once.

Page No.	
Date	

Syntax :-
(ANSI)
SELECT COL/Exp/*
FROM TABLE1 ALIAS JOIN TABLE2 ALIAS JOIN ..
TABLE N ALIAS.
ON SELF JOIN CONDITION
[WHERE FILTER CONDITIONS];

Q.1) Write SQL to find all the employee name and their manager name.

SELECT E1.ename, M1.ename
FROM emp E1, emp M1
WHERE E1.mgr = M1.empno;

EMP E1

empno	ename	mgr
01	SMITH	05
02	SCOTT	03
03	KING	NULL
04	ADAMS	05
05	ALLEN	02

EMP M1

empno	ename	mgr
01	SMITH	05
02	SCOTT	03
03	KING	NULL
04	ADAMS	05
05	ALLEN	02

Result of join

E1. empno	E1. ename	E1. mgr	M1. empno	M1. ename	M1. mgr
01	SMITH	05	05	ALLEN	02
02	SCOTT	03	03	KING	NULL
04	ADAMS	05	05	ALLEN	02
05	ALLEN	02	02	SCOTT	03

Single Row Function

The function which will allow n no of input and executes, finally which will give the n no of o/p wht I/P are known as SRF.

It executes Row by Row.

RULES OF SRF :

- 1] The SRF will allow multiple argument.
- 2] The SRF will not ignore the null values.
- 3] In where clause we can easily access all the SRF.
- 4] It is not possible to access the SRF in having clause.
bcz the execution process is different for both
having execute row by row group by group where
SRF executes Row by Row.
- 5] In the SELECT clause we can easily access SRF with an any general column.
- 6] There are the few SRF which are going to execute without using any argument.
Ex :- SYSDATE, CURRENT_DATE, SYSTIMESTAMP

Types of SRF :

- 1] TEXT OR STRING FUNCTIONS
- 2] DATE FUNCTIONS
- 3] NUMBER FUNCTIONS
- 4] CONVERSION FUNCTION
- 5] GENERAL FUNCTION
- 6] WINDOWS FUNCTION
- 7] PSEUDO COLUMNS.

1) Text or string functions :-

1) UPPER	5) REVERSE	9) LPAD	13) LENGTH
2) LOWER	6) REPLACE	10) RPAD	
3) INITCAP	7) SUBSTR	11) RTRIM	
4) CONCAT	8) INSTR	12) LTRIM	

1) UPPER Functions :

Used to convert string into upper case.

2) Lower Functions :

Used to convert string into lower case.

3) INITCAP Function :

It is used to convert first letter / character into uppercase and remaining lowercase.

Syntax :-

UPPER / LOWER / INITCAP ('STRING' / COL / FUN)

EQ :- SELECT ENAME, LOWER(ENAME), INITCAP(JOB)
FROM emp;

Result :	ENAME	LOWER(ENAME)	INITCAP(JOB)
	SMITH	smith	CLERK

4) CONCAT function :-

This funⁿ is used to merge any two given string

Syntax : CONCAT ('string' / COL, 'string 2' / COL)

Ex:- SELECT CONCAT (Ename, Sal)
FROM emp;

op: Smith 800
Allen 1600

Note: We cannot give space betn two col but we gave
space in ||

Concat operator (||)

SELECT 'HI' || Ename || " " || 'good morning'
FROM emp;

o/p:- HI Smith good morning.

5] Reverse Function:

It is used to Reverse given string

Syntax: SELECT REVERSE (Ename)
FROM emp;

6] Length Function:

It is used to count the no. of character present in the
given string.

Syntax: SELECT LENGTH ('string / col) F

Ex: SELECT LENGTH ('PHONE')
FROM DUAL;

Q. Write SQL to get all the details of emp the emp who earning 3 digit sal.
→
SELECT *
FROM EMP
WHERE LENGTH (SAL) = 3;

7] SUBSTR Function

This function is used to get the part of string from the given string

Syntax: SUBSTR ('string' | col, position, [length])

Ex:- SELECT SUBSTR ('@SPIDERS', 5, 2)
FROM DUAL;

O/P: PE

Ex:- SELECT SUBSTR ('@SPIDERS', 5)
FROM DUAL;

O/P: DERS

Ex:- SELECT SUBSTR ('MALAYALAM', 5, 8)
FROM DUAL

O/P: YALAM → Print space also if char are not enough

Ex: SELECT SUBSTR ('@SPIDERS', -3, 2)
FROM DUAL

O/P: ER

Note: The position value shouldn't be 0

- we can also use the negative position value
- length is optional part.
- The default value for length is remaining no. of char from position.

8] INSTR FUNCTION :-

This funⁿ is used to get the posn of an any char in the given string.

Syntax: INSTR('string' / col, 'character', position, [occurrence])

Ex: MALAYALAM

1) SELECT INSTR('MALAYALAM', 'V', 1)

FROM DUAL;

O/P → 5

2) SELECT INSTR('MALAYALAM', 'A', 1, 3)

FROM DUAL;

O/P → 6

3) SELECT INSTR('MALAYALAM', 'A', -1, 2)

FROM DUAL;

O/P → 4

1- Display all the players middle name.

→ SELECT SUBSTR(PNAME, INSTR(PNAME, ' ', 1, 1), INSTR(PNAME, ' ', 1, 1) - INSTR(PNAME, ' ', 1, 1))

FROM KKT;

Q - M S DHONI

O/P

→ S

HARD HITTING PANDYA

→ HITTING

K L RAHUL

→ L

VIRAT ANUSHKA KOHLI

→ ANUSHKA.

OR :-

```
SELECT SUBSTR ( SUBSTR ( PNAME, INSTR ( PNAME, ' ', 1, 1 ) + 1 ),
                1, INSTR ( SUBSTR ( PNAME, INSTR ( PNAME, ' ', 1, 1 ) + 1 ),
                            ' ', 1, 1 ) )
FROM KKAT ;
```

9] Replace Function

This function is used to replace any character or the string from the original string.

Syntax :- replace ('string' / col, 'character' / 'string', [new c/s])

```
Ex: - Replace ( 'aspiders', 'ders', 'zzzz' )
      FROM DUAL ;
```

// aspizzzz

```
Ex: SELECT REPLACE ( 'aspiders', 's' )
      FROM DUAL ;
```

// aspider

```
Ex: SELECT REPLACE ( 'aspiders', 's', 'z' )
      FROM DUAL ;
```

// azpiderz

```
Ex: SELECT REPLACE ( 'aspiders', 'a', 'j' )
      FROM DUAL ;
```

// jspiders

① - Ex: How to replace the second a as a z (MALAYALAM)

```
SELECT REPLACE ( 'MALAYALAM', 'a', 'z', 2 )
```

Shortest method →

```
SELECT SUBSTR ( 'MALAYALAM', 1, 3 ) || 'z' || SUBSTR ( 'MALAYALAM', 3 )
FROM DUAL ;
```

// MALZYALAM

Q ⇒ WHAT Replace the second A from all the places.

```
SELECT SUBSTR(PNAME, INSTR(PNAME, 'A', 1, 2) - 1)
      || 'Z' || SUBSTR(PNAME,
      INSTR(PNAME, 'A', 1, 2) + 1)
FROM PLC;
```

Table struc...
MAHARASHTRA
KARNATAK
HYDRABAD
HARIYANA

Q ⇒ WHAT Replace the last 4 digit of mobile number as *.

```
SELECT SUBSTR(MNO, 1, 6) || '****'
FROM person;
```

OR

```
SELECT SUBSTR REPLACE(MNO, SUBSTR(MNO, -4), '****')
FROM person;
```

numbers
1234567890
9876543212
4455668879
3265987412

10] LPAD and RPAD Function :

This functions are used to add the string within the specified length.

Syntax:- LPAD/RPAD ('STRING' / COL, LENGTH, 'NEW STRING')

```
EX:- SELECT LPAD('PIDER', 7, '*')      // **PIDER
      FROM DUAL;
```

```
EX:- SELECT LPAD('PIDER', 2, '*')      // PI → data loss
      FROM DUAL;
```

```
EX: SELECT RPAD('PIDER', 8, '*')      // PIDER ***
      FROM DUAL;
```


17 LTRIM and RTRIM FUNCTION :-

This function is used to remove / trim the extra spaces from the given string.

Syntax:- LTRIM / RTRIM ('STRING' / COL, 'C' / 'S')

↓
↓
 char or string

Ex: SELECT RTRIM ('SQL ')

FROM DUAL; // SQL

Ex: SELECT LTRIM (' SQL')

FROM DUAL; // SQL

18]

2] Number Function

- 1] MOD 2] ROUND 3] TRUNC 4] SQRT 5] POWER 6] ABS

1] MOD Function :-

This funⁿ is used to get the remainder value for any two given number.

Syntax: MOD (dividen, divisor)

Ex:- SELECT MOD (4,2)

FROM DUAL; // 0

Ex: SELECT MOD (5,2) // 1

FROM dual;

2] Round function :

This funⁿ is used to round up given value into its nearest value

Syntax: ROUND(DECIMAL)

Ex: SELECT ROUND(7.4) O/P = 7
FROM DUAL ;

Ex: SELECT ROUND(7.5) O/P = 8
FROM DUAL ;

3] TRUNC Function :-

This function is used to round off an any given value given to its lowest value.

Syntax: TRUNC(DECIMAL)

Ex: SELECT TRUNC(7.9) O/P = 7
FROM DUAL ;

4] SQRT

This funⁿ is used to get the square root value of any given no.

Syntax: SQRT(NUMBER)

Ex: SELECT SQRT(8)
FROM DUAL ; O/P = 2.82842712

5] power Function

This funn is used to get exponential value.

Syntax: POWER (BASE, RAISE)

EX:- SELECT POWER (2, 3)
FROM DUAL; // 8

6] ABS Function

This funⁿ is used to get absolute value of any number.

Syntax: ABS (Positive / negative number)

EX:- SELECT ABS (-10) // 10
FROM DUAL;

DATE FUNCTIONS

1] SYSDATE FUNCTION

This function is used to get the present date.

Syntax:- SYSDATE

2] CURRENT_DATE Function

This funⁿ is also used to get present date.

Syntax: CURRENT_DATE

3] SYSTIME STAMP FUNCTION :

This funn is used to get present date with time zone

Syntax - systime-stamp

EQ: SELECT sysdate, current_date, systimestam FROM DUAL;

4] MONTHS-BETWEEN FUNCTION

This function is used to get the no. of months betn an and given 2 days. dates.

Syntax: MONTHS-BETWEEN (DATE1, DATE2)

EQ:- SELECT MONTHS-BETWEEN (SYSDATE, Hiredate)
FROM emp;

5] ADD-MONTHS FUNCTION

This function is used to add the no. of months for given month. date.

Syntax:- ADD-MONTHS (DATE, NUM OF MONTHS);

EX: SELECT ADD-MONTHS (SYSDATE, 3)
FROM DUAL;

11 JUL-26

6] LAST-DAY FUNCTION

This function is used to get the last day of the last date for any given date.

Syntax:-

EX: SELECT LAST-DAY (SYSDATE)
FROM DUAL;

CONVERSION FUNCTIONS

- 1] TO-CHAR
- 2] TO-NUMBER
- 3] TO-DATE

1] TO-CHAR

This funⁿ is used to convert an any given date into string format.

Syntax: TO-CHAR (DATE, 'FORMAT')

```
SELECT TO-CHAR ( SYSDATE, 'YEAR')
FROM DUAL ;
```

// TWENTY TWENTY SIX

Format :-

YEAR → twenty twenty six

MON → AUG

MONTH → AUGUST

DAY → MONDAY

DD → 08

D → 4

MM → 08

HH12:MI:SS → 06:20:23

HH24:MI:SS → 18:20:23

Ex: SELECT TO-CHAR (SYSDATE, 'DAY') FROM DUAL ; // WEDNESDAY

2] To-number

This function is used to convert given string into number.

Syntax: `TO-Number ('String')`

3] To-date

This function is used to convert any given date into Oracle format.

Syntax: `TO-DATE ('DATE')`

Ex: - `SELECT TO-CHAR(TO-DATE('29-JAN-2006'), 'DAY')`
`FROM DUAL;` // SUNDAY

General Functions

1] NVL (Null Value Logic) :

This funⁿ is used to replace the null value with any specified number

Syntax: `NVL ( ARG1 / COL, ARG2 / COL )`

- If the first argument is null then it will give the second argument value
- If the first argument is a value then it will ignore the first argument value.

Ex :-

```
SELECT ENAME, SAL, COMM, SAL + COMM, SAL + NVL (COMM, 0)
FROM EMP;
```

2] NVL2 (Null Value Logic 2) :

This funⁿ is also used to handle the null values operation

Syntax: `NVL2 ( ARG1 / COL, ARG2 / COL, ARG3 / COL )`

3] COALESCE :

This funⁿ is used to consider the first not null argument from the multiple arguments.

Syntax: `COALESCE ( ARG1, ARG2, ARG3, --- ARG N )`

```
Ex: SELECT COALESCE (NULL, NULL, NULL, 100)
```

```
FROM DUAL;
```

O/P: → 100

```
Ex: SELECT COMM, MGR, SAL, COALESCE (COMM, MGR, SAL)
FROM emp;
```


Pseudo Columns

Pseudo columns are also known as the false column which are present in each and every table.

There are 2 types of pseudo columns.

1] Row id

2] Row num

1] Row id :-

- Row id is considered as the address of record.
- Row id is static in nature.
- Row id consist 18 digit hexadecimal numbers.
- Fetching the records by using the Rowid will consider as the fastest method to fetch the data.

2] Row num :-

- Row num is like the Serial no. of each and every record.
- The Row num will start from 1.
- The Row num will get increment by 1.
- Using Row num we can consider the fastest method to fetch the data from table.
- The Row num is dynamic in nature.
- We can convert the dynamic property of Row num as static by using alias name.

Q. Write the first 3 records from emp table

→ `SELECT * FROM (SELECT Emp.*, Rownum SlnO FROM Emp)
WHERE SlnO <= 3;`

WINDOWIS FUNCTION

This function are used to assign the ranking for the records based on the values.

There are 2 types of Window Functions :-

1) RANK()

2) DENSE-RANK()

Syntax: RANK / DENSE_RANK() OVER (ORDER BY col asc / desc,
[partition by col]);

partition by first then order by

1) RANK()

It is one type of windows funⁿ which is used to assign the rank value for the record based on the value which are present in table.

- For the repeated element the rank funⁿ will assign the proper ranking but for next element it will skip the ranking. This is the drawback of RANK().

2) DENSE-RANK()

- It is similar to rank funⁿ
- The dense rank function will not skip any rank value if the repeated elements are present.

Note

In the windows funⁿ there is the partition by clause which is very similar to group by clause

EX: SELECT * FROM (SELECT emp.*, DENSE-RANK() OVER (PARTITION BY
dno ORDER BY sal desc) DR FROM emp)

WHERE DR = 2;

DDL (Data Definition Language)

This statement is used to create or modify or destroy an any database object.

The database objects are :

- | | |
|------------|--------------------------------|
| 1] TABLE | 4] PROCEDURES |
| 2] VIEWS | 5] UDF (used define functions) |
| 3] INDEXES | 6] TRIGGERS. |

These are 6 commands in DDL :

- | | | |
|-----------|-------------|--------------------|
| 1] CREATE | 3] TRUNCATE | 5] ALTER / MODIFY. |
| 2] RENAME | 4] DROP | |

1] TABLES

It is the combination of rows and columns.

1] CREATE

This command is used to create an any database object

• Create table :-

Syntax : create table table_Name

(col1 datatype [constraints],

col2 datatype [constraints],

-----,

coln datatype [constraints],

[constraint REF_Name constraint_Name (col_Name)

REFERENCE parent_Table_Name (col_Name)]

);

Note

- To create table we need table name and all the columns.
- We must have to validate schema by using datatype and constraints, constraints are optional.
- The table name or col name shouldn't start with any no. or any special char.
- The table name or col name should be the single string.
- In the same table it is not possible to take same columns.
- In a same data base it is not possible to create same table.

Q. WROTE create the student table with following columns.

STD			
SID	Sname	mno	Gmail
NUM(3)	VARCHAR(30)	NUM(10)	VARCHAR(50)
		(C) → check constraint	• com → validate by using.

→ CREATE TABLE STD

(SID NUMBER(3) UNIQUE,
 Sname VARCHAR (30),
 Mno number (10) check (length (mno)=10),
 mail varchar (50) check (mail like '%.com')
);

Q - WROTE create the teachers tables with following column.

TID	Tname	Tage	TSal	Tmno	Taddress
num(3)	varchar(30)	NUM(3)	num(8)	num(10)	VAR(20)
		> 18	> 0	= 10	Default
				starts with 9 only	pune.

→ CREATE TABLE Trained

(TID number(3),

Tname varchar(30),

Tage num(3) check (age > 18)

Tsal num(8) check (|sal| > 0)

tmno num(10) check (LENGTH(tmno) = 10 AND

tmno LIKE '9%');

taddress VARCHAR2(20) DEFAULT 'Pune');

Q. What create the flight table by using the following columns.

Flight

FID	Fname	Cname	Ddate	Atime	Dtime
NUM(3)	VARCHAR(50)	VARCHAR(50)	DATE	VARCHAR(20)	VARCHAR(20)
PK	Repetition draw				
	NOT NULL	NOT NULL	NOT NULL	NULL	NOT NULL

→ CREATE TABLE FLIGHT

(FID NUMBER(3) PRIMARY KEY,

FNAME VARCHAR(50) NOT NULL,

CNAME VARCHAR(50) NOT NULL,

ATIME VARCHAR(20) NULL,

DTIME VARCHAR(20) NOT NULL,

DDATE DATE);

IF we directly use PK
then reference will
not generated

So it's the drawback

OR

CREATE TABLE FLIGHT

(FNO NUMBER,

FNAME VARCHAR(50) NOT NULL,

CNAME VARCHAR(50) NOT NULL,

DDATE DATE,

ATIME VARCHAR(20) NOT NULL,

DTIME VARCHAR(20),

CONSTRAINT FNO_PK PRIMARY KEY (FNO));

in that above case
use this.

RENAME

This command is used to change the name of existing object.

Syntax: `RENAME OLD_TABLE_NAME TO NEW_TABLE_NAME;`

Ex: `RENAME Emp TO Employee;`

Ex: `RENAME Dept TO DEPARTMENT;`

TRUNCATE

This command is used to completely delete all the records from the table permanently.

Syntax: `TRUNCATE TABLE TABLE_NAME;`

Ex: The truncate command will not support where clause, and also will not support rollback.

Ex: `TRUNCATE TABLE SALGRADE;`

`SELECT * FROM SALGRADE;`

Rollback → to get all data again.

The truncate command is not possible to apply on the parent table.

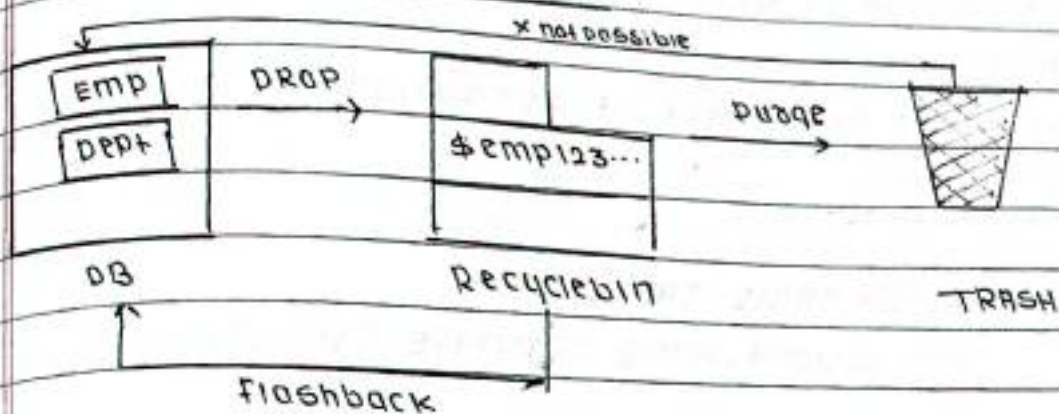
We can apply the truncate on the parent table after removing the connection (Remove connection using alter).

DROP

This command is used to delete the table from database to recycle bin.

- After deleting the table we can able to get it back from recycle bin with the help of Flash back command.

• If we want to delete the table even from the recycle bin we can use purge command.



DROP Syntax :- DROP TABLE TABLE_NAME;

Flashback Syntax : FLASHBACK TABLE TABLE_NAME TO BEFORE DROP;

PURGE Syntax : PURGE TABLE TABLE_NAME;

OR

DROP TABLE TABLE_NAME PURGE;

TO CHECK TABLE FROM BIN : SELECT * FROM Recycle bin;

• It is not possible to apply directly the drop command on parent table, If we remove the connection then we can easily apply drop command from table.

TO CHECK TABLE FROM DB : SELECT * FROM TAB;

■ ALTER

→ It means modification

This command is used to modify the structure of an any existing object.

These are 7 diff type of operations.

1] Add column

Syntax: ALTER TABLE TABLE_NAME
 ADD COLUMN-NAME DATATYPE [CONSTRAINTS];

Ex1:- ALTER TABLE EMP FLIGHT
 ADD A MNO Number NOT NULL;

Ex2:- ALTER TABLE EMP
 ADD MNO Number NULL;

Ex3:- ALTER TABLE EMP
 ADD A PHAR Number NOT NULL;

ERROR:- PUE TO TABLE is not empty.

- Adding mandatory col for the table which is not empty is not possible
- Adding mandatory col for the table which is empty is possible.
- Adding the optional col is possible for all the table.

2] Remove column And DROP Column.

Syntax: ALTER TABLE TABLE_NAME
 DROP COLUMN COLUMN-NAME.

Ex: ALTER TABLE emp
DROP COLUMN mno;

Note

We can drop any column but dropping the primary key from parent table is not possible until we remove the connection

5] RENAME COLUMN

Syntax: ALTER TABLE TABLE_NAME
RENAME COLUMN OLD_COL_NAME TO NEW_COL_NAME;

Ex:- Alter table emp
RENAME COLUMN empno TO EID;

6] ADD CONSTRAINT

Syntax: ALTER TABLE TABLE_NAME
ADD CONSTRAINT REF_NAME CONSTRAINT_NAME (COLNAME);

Ex: Alter table ABCD
ADD CONSTRAINT NAME-PK PRIMARY KEY (NAME);

7] DROP CONSTRAINT

Syntax: ALTER TABLE TABLE_NAME
DROP CONSTRAINT REF_NAME;

Ex: ALTER TABLE ABCD
DROP CONSTRAINT NAME-PK;

6] Change datatype

Syntax: ALTER TABLE TABLE-NAME
MODIFY COL_NAME NEW DATATYPE ;

EX: ALTER table Flight
MODIFY Mno date ;

EX: ALTER table EMP
modify Sal Date.

ERROR: column to be modified must be empty to change datatype

7] Make col as NULL OR NOTNULL / Convert NULL to Not NULL VISE V

Syntax: ALTER table table_name
MODIFY COL PRESENT_DATATYPE NULL / NOT NULL ;

EX: ALTER table Flight
MODIFY MNO DATE NULL ;

DML (Data Manipulation Language)

- This statement is used to insert and delete and modify the data which is present in the table
 - The DML is also known as auto commit statement.
 - The DML is also known as non auto commit statement.
- In the DML there are 3 commands:

1] DELETE

This command is used to delete any specific record from table.

Syntax: DELETE FROM TABLE_NAME
[WHERE CONDITION];

2] UPDATE

This command is used to change the values from the table.

Syntax: UPDATE TABLE_NAME
SET COL1 = V1, COL2 = V2, ... COLN = VN
[WHERE CONDITION];

3] INSERT

This command is used to insert the records into specific table.

Syntax: INSERT INTO TABLE_NAME VALUES
(V1, V2, V3 ... VN);

OR

INSERT INTO TABLE_NAME VALUES
(&COL1, &COL2, ... &COLN);

→ most of the cases we use this cmd.

specific value will get delete using Null and update bcz if we use delete cmd it will delete the record not the specific val.

Page No	
Date	

Transaction control Statement (TCL)

This statement is used to control the transaction in the database.

These are three commands in TCL

1] commit

2] Roll Back

3] save point.

1] commit.

This command is used to save the transaction in db.

Syntax: commit;

2] Roll Back

This command is used to get back the transaction before commit.

Syntax: Rollback;

3] Save point

This command is used to control the transactions by using multiple levels.

Syntax: Savepoint savepoint_Name;

To control the transaction by using levels

Syntax: Rollback to savepoint_Name;

DCL (Data Control Statement / Lang)

This statement is use to control the data flow betn multiple users.

There are 2 commands in DCL :

1] Grant

2] Revoke

1] Grant

This command is used to give the access to any table to user.

```
Syntax: GRANT SQL-commands  
ON TABLE-NAME  
TO USER-NAME ;
```

2] Revoke.

It is used to get back the access from the users.

```
Syntax: REVOKE SQL-Command  
ON TABLE-NAME  
FROM USER-NAME ;
```

Note : To check all the access

```
SELECT * FROM user_TAB_PRIVS;
```

```
SELECT --- AS VIEWER (READ)
```

```
DELETE / UPDATE / INSERT --- AS EDITOR (WRITE)
```


NORMALIZATION

It is process of dividing the bigger table to n no. of smaller table in order to remove the redundancy and anomalies

It is process of dividing the bigger table into n no of small tables until to get a perfect table

The normalization is depends on key attributes and the functional dependancy.

1] Key Attributes

In SQL there are around 10 key attributes.

1] Key attribute

2] non-key attribute

3] prime-key attribute (primary)

4] non-prime key attribute

5] composite attribute

6] super key attribute

7] composed attribute

8] derived attribute.

9] Single valued and multi-valued attribute

10] foreign key attribute.

1] Key attribute :-

The attribute which are used to identified the unique record.

2] non-key attribute :-

accept all the key attribute except remaining attribute are called non-key attribute

3] prime key attribute

among all the key attribute the one attribute will be chosen to identify the unique record is called prime key attribute.

4] Non-prime key attribute

Except all the prime key attribute the remaining key attribute is called as non-prime key attribute.

5] composit attribute.

The set of all the non key attribute is known as composite.

Ex: $B = \{ \text{Ename, age, Exp, add} \}$

6] super-key attribute.

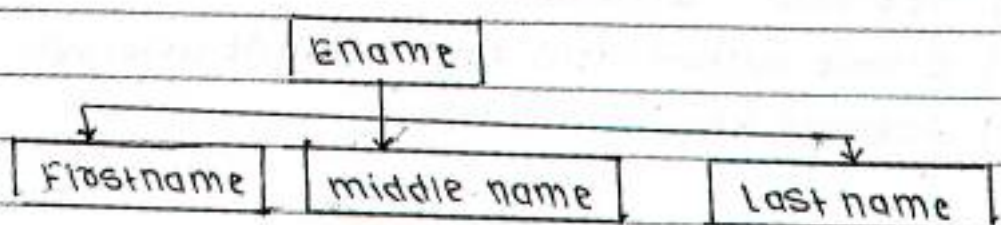
The set of all the key attribute is known as super key.

Ex: $A = \{ \text{Eno, ADHAR, VID, mno} \}$

7] composed attribute.

The attribute which we can easily divide into multiple attribute known as composed.

Ex:



8] derived attribute.

The attribute which are calculated using other attribute the attribute.

9] Single value / And multivalued attribute.

Single value : the attribute which will having single value
 Multi value : the attribute which will having multiple value.

ENO	ENAME	SAL	MNO	ADHAR	VID	AGE	EXP	Address
KA			KA	KA	KA			
	NKA	NKA				NKA	NKA	NKA
PKA								
			NPKA	NPKA	NPKA			
	CA					CA	CA	CA
						DA	DA	
SVA	MVA	MVA	MVA	SVA	SVA	SVA	MVA	MVA

10] Foreign Key attribute :

It is used to connect tables after data is divided into multiple tables to remove redundancy.

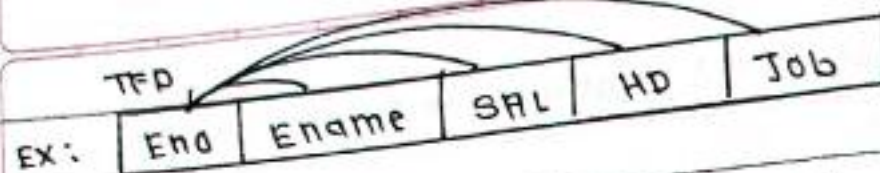
Functional dependency in SQL.

Types :-

1] Total functional dependency

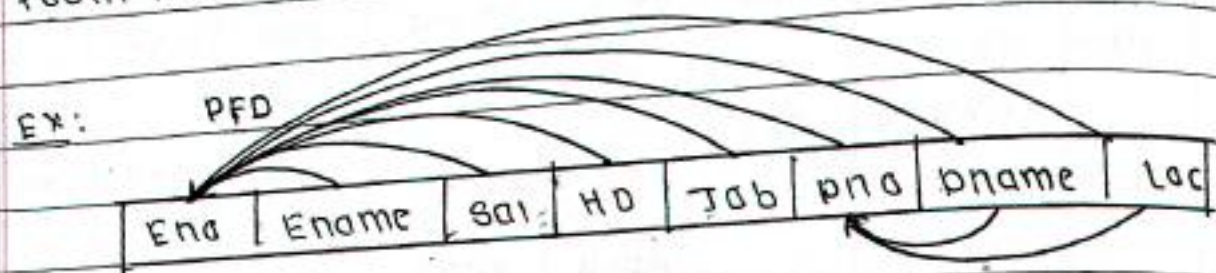
All the attributes from the table if they depend on only one key attribute this type of dependency is called as total functional dependency.

If the table consists of only TFD including no repeated record in single cell attribute data this type of table is known as perfect table.



2] Partial Functional Dependency :

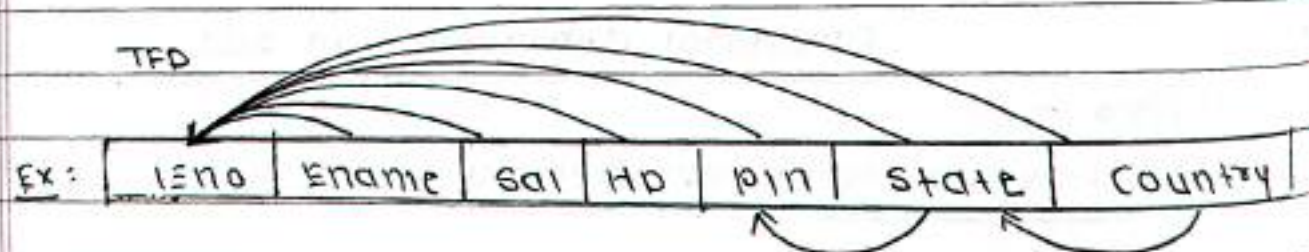
The attributes from the table if they depend partially from multiple key attribute is called PFD.



If the table consist of partial functional dependency this type of table are known as imperfect table.

3] Transitive function dependency (Indirect type of dependency)

The attribute which depends on another key attribute and that attribute itself depends on another key attribute is called TFD.



If the table consist of transitive functional dependency is also known as imperfect table.

Normalization :-

Dividing bigger table into n no. of smaller tables to convert imperfect table into perfect table is called normalization

Normalization done in 3 stages

- 1) 1NF
- 2) 2NF
- 3) 3NF
- 4) 4NF (Boyce Codd Normal Form)

1) 1NF (First Normal Form) :-

We can consider for the table is in 1NF, if the table is not having any repeated records.

If the table is having single value in a cell.

						STP		
SID(PK)	SName	S1	S2	S3		SID	SName	Sub
S1	SHAM	SOL	JAVA	WJT	⇐	S1	SHAM	SOL, JAVA
S2	RAJU	SOL		MT		S2	RAJU	SOL, MT
						S3	SHAM	MT

2) 2NF (Second Normal Form) ;

We can consider the table is under the 2NF

1) The table should be in 1NF

2) The table shouldn't consist any partial functional dependency

Emp (PK)	ENAME	SAL	HP	JOB	PNO	PNAME	LOC	
Emp (PK)	ENAME	SAL	HP	JOB	PNO / FK	PNAME	LOC	PNO / PK

T1
Perfect

T2
Imperfect

depends on only one attribute

depends on 2 attribute

2) 3NF (third normal form)

For the table we can consider is in 3NF, the table should be in 2NF.

The table should not consist of any transitive functional dependency

Emp(PK)	Ename	Sal	HD	Job	DAO	Dname	loc
				Pin	State	Country	

Emp(PK)	Ename	Sal	HD	State	Country ^{FK}	Pin(PK)
---------	-------	-----	----	-------	-----------------------	---------

Pin(PK)	State	Pin(FK)	Country
---------	-------	---------	---------